SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1

Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

Scrum with Lean flow controls, planned and tracked in Jira

Student Name	Ashira Bridget Susan S
Register Number	192524316
Programme and Section	CSA1021 – Software Engineering
Nature of Assignment	Individual | Total marks: 50
Submission Date	13/11/2026

Course outcome. CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Honesty statement. All stakeholder responses, sprint events, board states and indicators in this report are SIMULATED planning evidence and are labelled as such. Jira-style figures are illustrative mock-ups, not screenshots of a live project, and nothing here is a record of actual team delivery.

Contents

Executive summary **3**
Reading conventions 3
Task 1 – Investigate the Problem and Define Delivery Outcomes (5 marks) **4**
1.1 Stakeholders and investigation questions 4
1.2 Evidence from stakeholders (SIMULATED) 4
1.3 Problem statement and product goal 5
1.4 First-release scope and exclusions 5
1.5 Planning assumptions: workload, service levels and thresholds 6
1.6 Measurable delivery-success criteria 6
1.7 Feasibility 7
Task 2 – Design and Justify the Project Management Approach (6 marks) **8**
2.1 Two delivery approaches 8
2.2 Selection 8
2.3 Tailored operating model 9
2.4 Lean practices applied 10
2.5 Project-management tool investigation and selection 10
Task 3 – Construct a Prioritised Product Backlog in Jira (8 marks) **12**
3.1 Project set-up in Jira 12
3.2 Epics and the release decision 12
3.3 Prioritisation method 12
3.4 Backlog: stories, estimates, dependencies and owners 13
3.5 Acceptance criteria 15
3.6 Definition of Ready and Definition of Done 16
3.7 Decisions where stakeholder requests competed 17
Task 4 – Capacity-Based Release Plan and Delivery Workflow (7 marks) **18**
4.1 Capacity assumptions 18
4.2 Sprint goals and selected work 18
4.3 Dependencies and constraints 19
4.4 Development workflow in Jira and its separation from the order lifecycle 20
4.5 Work-in-progress limits and flow policies 20
4.6 How the selected features achieve the product goal 21
Task 5 – Simulate Delivery and Validate the Design (7 marks) **22**
5.1 Prediction before the simulation 22
5.2 Event log and Jira updates (SIMULATED) 22
5.3 Delivery indicators (SIMULATED) 23
5.4 Interpretation 24
5.5 Limitation found and improvement for the next sprint 24
Task 6 – Redesign Delivery Under Changed Conditions (12 marks) **25**
6.A Diagnose the impact (2 marks) 25
6.B Measurable redesign objectives (2 marks) 26
6.C Revised strategy (4 marks) 27
Options considered 27

Scaled Agile practices adopted 27
Revised backlog and ownership 28
Responsibilities, Lean practices and Jira artefacts 28
6.D Validate the redesign (2 marks) 29
6.E Defend the decision (2 marks) 30
Task 7 – Individual Technical Reflection (5 marks) 32
Appendix A – Mapping to the assignment requirements 33
Appendix B – Creating the real Jira project and screenshots 33
Appendix C – Acknowledgement of sources and tools 33
Appendix D – Preparation for the verification discussion 34

Executive summary

A retailer selling through a website, a mobile app and twelve stores wants an integrated order fulfilment platform that reduces failed orders, slow refunds and unclear order ownership. Management asked for all nine modules in one eight-week release. This report designs, simulates and validates an Agile delivery strategy that instead protects **one complete order-to-delivery-to-refund journey**, because a broad release risks leaving every workflow incomplete and a storefront-only release would not reduce failed or delayed orders.

The strategy is **Scrum with Lean flow controls and a few XP practices**, run on **Jira**. It uses four two-week sprints, a 59-point prioritised backlog of 17 stories in four epics, explicit WIP limits (4 / 2 / 2), a payment stub and shared contract to remove the dependency on the late gateway, and a weekly 90-minute stakeholder window. A simulated first sprint (13 of 16 points, 3 blocked story-days, 2 reopened stories) showed that the real constraint was a missed story dependency and an unready story rather than the QA queue I had predicted. The plan was revised accordingly. When two regional warehouses, a second team and the loss of the payments engineer arrived, the strategy was redesigned around shared contracts, an integration lane, region rules held as configuration and explicit scope deferral, and was compared with the original strategy under identical simulated indicators.

Table 1. Report at a glance (all sprint figures are simulated).

Item	Value
Release length / sprints	8 weeks / four two-week sprints (festival date fixed)
Product backlog (first release)	17 stories, 59 points, 4 epics; 2 analytics/ML items deferred
Original capacity (Sprint 1)	7 people x 10 days x 6 h = 420 h nominal; 336 h after 20% reserve
Sprint 1 (simulated)	13 of 16 points (81.3%); 3 blocked story-days; 2 reopened stories; mean cycle time 7.75 days
Revised capacity (Sprints 2-4, simulated change)	1,448 effective hours, about 1.44x the original three-sprint total of 1,008 h, not 2x
Revised release (Sprints 2-4)	51 committed points across two teams, 3 stretch points; OFP-13, 14 and 23 (7 points) newly deferred
Revised Sprint 2 (simulated)	14 of 16 points (87.5%); 1 blocked story-day; 1 reopened story
Deadline credibility	Remaining 37 points vs forecast 35-39: feasible only with a pre-agreed contingency ladder

Reading conventions

- **SIMULATED** marks any stakeholder response, event, board state or indicator that was constructed for this assignment. **Assumption** marks a planning figure that I chose and have not measured.
- **Figures that look like Jira** are mock-ups drawn to show the structure of my project (epics, priorities, estimates, WIP limits). The CSV import file supplied with this report (OFP_jira_import.csv) lets the same project be created in Jira so the figures can be replaced by genuine screenshots (Appendix B).
- Story points are relative size, not hours. Hour figures appear only in capacity arithmetic and in a bottom-up sanity check; they are never used to convert points.
- Deterministic rules (for example a fixed low-stock threshold) are kept separate from trained machine-learning forecasting, which is a proposed extension and not part of the first release.

Task 1 – Investigate the Problem and Define Delivery Outcomes (5 marks)

1.1 Stakeholders and investigation questions

Six user groups and the development team shape the platform. Customers need trustworthy order status; pickers need accurate pick lists; support agents need readable order and return status; finance reviewers need auditable refunds; store managers need stock they can rely on; administrators need controlled permissions; engineers need traceable links between orders, payments and releases. I framed five questions, each tied to a decision it will inform.

Table 2. Investigation questions.

#	Investigation question	Why it matters	Method (simulated)	Decision it informs
Q1	Why does stock shown as available fail at picking time?	Failed orders are the first operational problem.	Picker interview; review of pick-failure log	Reservation at checkout with expiry (OFP-4)
Q2	Which rules decide who may approve a refund and up to what amount?	Slow and unauditable refunds are the second problem.	Finance and support interviews; process review	Role- and amount-based approval (OFP-8)
Q3	What must happen when the payment gateway is slow, unavailable or called twice?	Sandbox access is late (week 4); double charges are unacceptable.	Developer interview; gateway document review	Idempotency key and local payment stub (OFP-5)
Q4	Which order milestones must customers and support agents see, and which must stay internal?	Unclear ownership drives repeat calls.	Customer and support interviews	Permitted milestones only (OFP-12, OFP-16)
Q5	How much can the team deliver given uncertain capacity and 90 minutes of stakeholder time per week?	Scope must fit a fixed date.	Team planning session; calendar review	Four-sprint plan, WIP limits, stakeholder cadence

1.2 Evidence from stakeholders (SIMULATED)

Label: No real stakeholders were available, so the responses below are simulated and written to be plausible for a retailer of this kind. They are used as planning inputs, not as research findings. The scores in Figure 1 are likewise simulated.

Table 3. Simulated stakeholder findings and the planning decision each produced.

Stakeholder	Simulated finding	Planning decision
Customer	"Tracking says Processing for days; I do not know who owns my order or when it ships."	Show only permitted milestones (Paid, Picked, Dispatched, Delivered) with plain text, no internal notes (OFP-12).
Warehouse picker	"Pick lists contain items another order already took; I walk to an empty bin."	Reserve stock at checkout with a 15-minute expiry and show reservation on the pick list (OFP-4, OFP-10).
Support agent	"Customers phone again and again because refund status is invisible; I cannot find an order quickly."	Prioritise order lookup with visible return and refund status (OFP-16).
Finance reviewer	"Large refunds are approved by chat message and I cannot reconcile payments to orders."	Role- and amount-based approval with an audit record (OFP-8); reconciliation export as a Should (OFP-9).
Store manager	"The app and the store disagree on stock; I override counts by hand."	Stock changes only through audited transitions; manual corrections need a reason and a human (OFP-4).
Developer	"Sandbox access arrives in week 4; we cannot test payments before then, and one engineer owns payments."	Build a payment stub and contract first (OFP-5); defer real integration to Sprint 3 (OFP-6); plan pairing on payments.

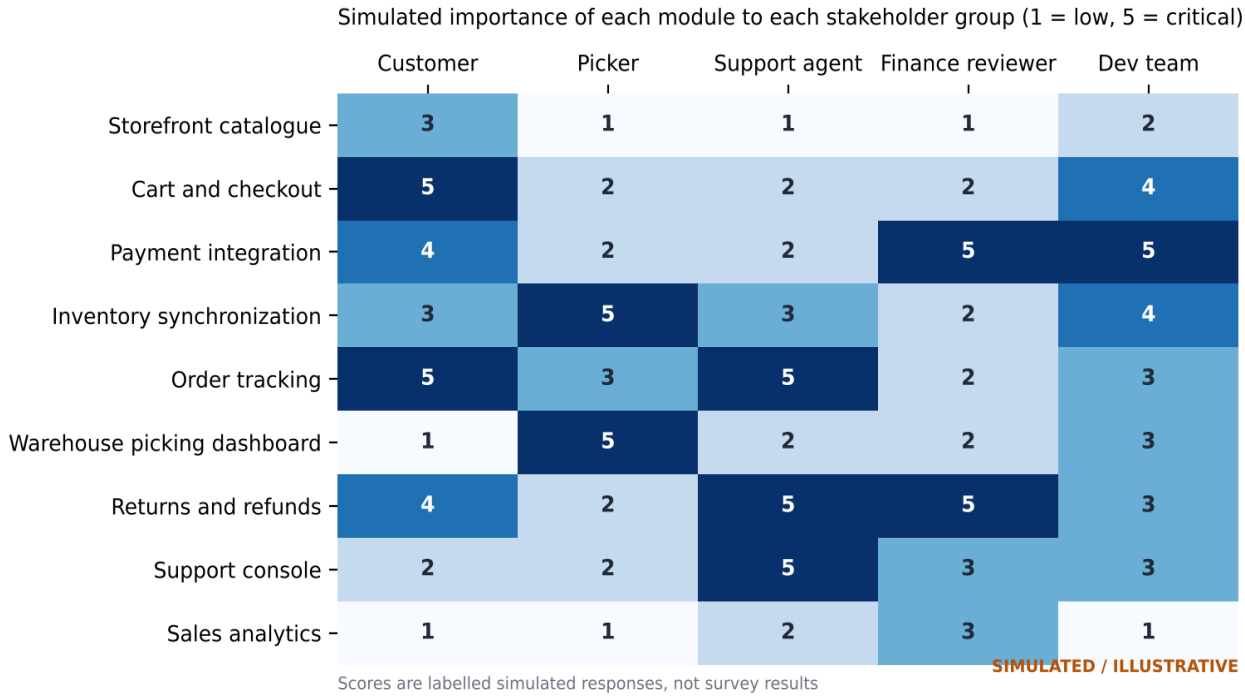


Figure 1. Simulated importance of each module to each stakeholder group (SIMULATED responses).

Reading Figure 1. Pickers rate inventory and picking highest, support agents rate tracking, returns and the support console, finance reviewers rate payments and returns/refunds, and sales analytics is low for every group. Cart/checkout, payment integration and returns/refunds are important to several groups at once, which makes them the backbone of the first release. Analytics is the only module nobody needs on day one.

1.3 Problem statement and product goal

Problem statement. Failed orders, slow and unauditable refunds and unclear order ownership arise because stock, payment, picking, tracking and refund steps are not connected by one trustworthy record. The team must choose a feasible eight-week release before a fixed festival-sale date despite unknown velocity, 90 minutes of stakeholder time per week and a payment sandbox that opens in week four.

Product goal. Make an order traceable from checkout to delivery, and where needed to an authorised refund, with stock that customers and pickers can trust.

1.4 First-release scope and exclusions

Table 4. Scope boundaries.

In the first release	Outside the first release (roadmap only)
Role-based sign-in; seeded catalogue and cart; persisted checkout order Stock reservation with expiry Idempotent payment request (stub first, then gateway sandbox) Pick list, dispatch and audited status transitions Customer tracking page with permitted milestones Return request; refund approval with audit Support lookup of order, return and refund status Reproducible build and an end-to-end journey test Rule-based low-stock alert (fixed threshold)	Real payment processing and live card handling Sales analytics dashboard (OFP-18) Demand forecasting and recommendations (trained ML, OFP-19, proposed extension) Multi-currency pricing, loyalty and promotions Native mobile app (the API and responsive web are the delivery channel) External ERP stock synchronisation (the reservation model stands in for it)

Table 5. Disposition of the nine intended modules. All nine remain on the roadmap; a menu entry does not make a module complete.

Module	Release decision	Reason	Story
Storefront catalogue	Retained (minimal)	Seeded products are enough to place orders	OFP-2
Cart and checkout	Retained	Starts the journey	OFP-2, 3
Payment integration	Retained (stub, then sandbox)	Needed to mark orders Paid and to refund	OFP-5, 6
Inventory synchronization	Retained as reservation and pick-level stock	External ERP sync deferred	OFP-4, 14
Order tracking	Retained	Customer and support priority	OFP-11, 12
Warehouse picking dashboard	Retained (pick list; scan is Should)	Picker priority	OFP-10, 13
Returns and refunds	Retained (single approval workflow)	Finance priority and journey end	OFP-7, 8, 9
Support console	Retained (lookup only)	Support priority; no case management	OFP-16
Sales analytics	Deferred	Needs trustworthy records first	OFP-18

1.5 Planning assumptions: workload, service levels and thresholds

Table 6. Explicit planning assumptions.

Assumption (all unmeasured)	Value used for planning and tests
Peak workload	1,200 orders per hour at festival peak; 120 concurrent checkouts (tests use 100 concurrent orders)
Sites	12 stores and 1 central warehouse in the original plan; 3 warehouses after the Day 4 change
Reservation expiry	15 minutes; released on cancellation or expiry
Return window	14 days from delivery
Refund approval	Always by a finance reviewer; refunds above INR 10,000 need a second approver; no automatic approval
Stock correction	Manual, with reason, role-checked and audited; never automatic
Status visibility	Order transitions visible to customer and support within 5 seconds in tests
Pick list generation	Under 3 seconds for up to 50 lines (test target)
Gateway behaviour	Timeout 10 seconds; 3 retries with back-off; sandbox from week 4
Low-stock rule	Deterministic: alert once when available stock is at or below a fixed per-item threshold. This is not a forecast.

Dashboard values, stock counts and screenshots in this report are illustrative evidence of displayed behaviour. They do not prove production accuracy, performance, live inventory control or secure payment handling; each claim is tied to a repeatable check in the success criteria below.

1.6 Measurable delivery-success criteria

Table 7. Success criteria. These are test targets, not claims of production achievement.

ID	Criterion	Acceptance condition (test target)	Check
SC1	Core journey	One synthetic order reaches Delivered and one return reaches Refunded, with an audit record for each valid transition	E2E script (OFP-17)
SC2	Stock integrity	100 concurrent test orders for a 10-unit item create at most 10 reservations and never a negative count	Concurrency test (OFP-4)
SC3	Payment idempotency	Ten retries with one payment key create one payment intent and one order	Replay test (OFP-5)
SC4	Refund control	Support and customer roles are rejected in 5 of 5 unauthorised-approval tests; every decision has an audit record	Negative tests (OFP-8)
SC5	Predictability	At least 80% of planned points done per sprint; at most 2 blocked story-days; at most 0.25 reopened stories per completed story	Jira sprint report and export
SC6	Reproducibility	A clean checkout and documented build reproduce the journey and keep records after restart	Fresh-machine run (OFP-15)

ID	Criterion	Acceptance condition (test target)	Check
SC7	Accessibility	Tracking and support status pages pass a keyboard-only and non-colour-status checklist (100%)	Checklist in Definition of Done

1.7 Feasibility

Table 8. Feasibility observations at planning level.

Dimension	Observation	Risk and response
Technical	A relational database with transactions, a REST API and a thin web UI are sufficient. Concurrency (reservation) and idempotency (payment) are the hard parts. The gateway can be replaced by a local stub with a fixed contract.	Late sandbox access: build stub and contract first. Single payments engineer: pair on payments.
Operational	Warehouse and finance stakeholders give 90 minutes per week. The reservation expiry rule, refund limits and permitted milestones need their decisions early.	Use a weekly clarification window; record decisions as acceptance examples; the Product Lead decides if the window is missed and flags it at the next review.
Financial	Planning cost uses an assumed blended INR 4,500 per person-day: 7 people x 40 days = 280 person-days = about INR 12.6 lakh. Development uses existing computers, free test accounts and a free sandbox. Tooling is free for up to 10 Jira users (verify current plan limits).	Any paid hosting or a Jira plan upgrade is estimated separately (it becomes relevant when the second team joins in Task 6).

Task 2 – Design and Justify the Project Management Approach (6 marks)

2.1 Two delivery approaches

Table 9. Comparison of the two approaches.

Dimension	Approach A: Scrum with Lean controls and XP practices	Approach B: Kanban flow with release milestones
Selecting work	Product Owner orders the backlog; team forecasts a sprint goal and pulls stories that fit capacity at planning.	Highest-value Ready item is pulled whenever a WIP slot opens; no fixed batch.
Coordinating	15-minute daily meeting, shared sprint board, weekly refinement, short API design sessions.	Daily flow review of the board, pull signals, explicit policies on column exit.
Reviewing	Sprint review with stakeholders every two weeks; retrospective; indicators (completion, cycle time).	Weekly service-delivery review with warehouse and finance; flow metrics (lead time, throughput, aging WIP).
Changing requirements	Change enters the backlog and is traded against the sprint goal at the next planning, with a rule for urgent items.	Change is absorbed continuously; expedite lane for urgent work.
Cross-functional work	Whole-team sprint goal pushes the seven roles to finish one slice together.	Roles specialise in columns; handovers are visible but not forced to converge.
Overheads	Planning (2 h), review, retrospective each sprint; about 8-10% of time.	Lighter ceremonies; needs strong discipline on policies and milestone tracking.
Likely limits and risks	No velocity yet, so first-sprint commitment may be wrong; a sprint boundary can delay urgent change.	Weak milestones can leave payment integration until late; stakeholders get fewer predictable review points; hard to defend a fixed date.

Approach A fits a release with a fixed end date, scarce stakeholders and cross-role stories. Approach B fits a continuous support stream. XP practices (acceptance examples, pairing on contracts, continuous integration, small slices) can be added to either.

2.2 Selection

Table 10. Weighted decision matrix (my scores 1-5; the weights are my judgement).

Criterion	Weight %	A: Scrum + Lean	B: Kanban
Predictable stakeholder review points (90 min per week)	20	5	2
Handling of changing requirements	15	3	5
Sequencing around gateway (wk 4) and scanner (wk 5) constraints	20	4	3
Visibility of waiting and WIP	15	4	5
Overhead for a seven-person team	10	3	4
Forecasting against a fixed festival date	20	5	3
Weighted score (max 5)	100	**4.15**	**3.50**

Selected: Approach A, Scrum with Lean controls (score 4.15 against 3.50). A sprint boundary gives warehouse and finance a predictable review point, a sprint goal keeps all seven roles on one end-to-end slice, and WIP limits borrowed from Kanban expose queues in review and test. Kanban's strength, continuous reprioritisation, is kept through an expedite rule. The weakness of Scrum, overcommitment before velocity exists, is handled by a deliberately provisional capacity ceiling (Task 4) that is revised after Sprint 1.

2.3 Tailored operating model

Table 11. Roles and decision rights. Engineers own completion evidence; the team shares the sprint goal.

Planning role	Owns	Decides
Product & Analysis Lead (PL)	Backlog order, acceptance examples, stakeholder window, escalations	Scope conflicts; what is deferred; accepts stories at review
Frontend engineer (FE)	Cart, tracking and support screens; accessibility checks	UI patterns; screen-reader and keyboard compliance
Order API engineer (API)	Checkout, status transitions, order contract	Order API design; integration owner of the order contract
Inventory & data engineer (INV)	Reservation, pick list, stock rules, reports	Stock rules; reservation behaviour under concurrency
Payments & security engineer (PSE)	Sign-in and permissions, payment idempotency, gateway, refund approval	Security design; what counts as sensitive data
QA engineer (QA)	Acceptance checks, test data, E2E script	Can reject any story that fails its acceptance criteria
DevOps engineer	Build, deploy, smoke checks; acts as flow coach, facilitating events	Pipeline; WIP-limit enforcement reminders

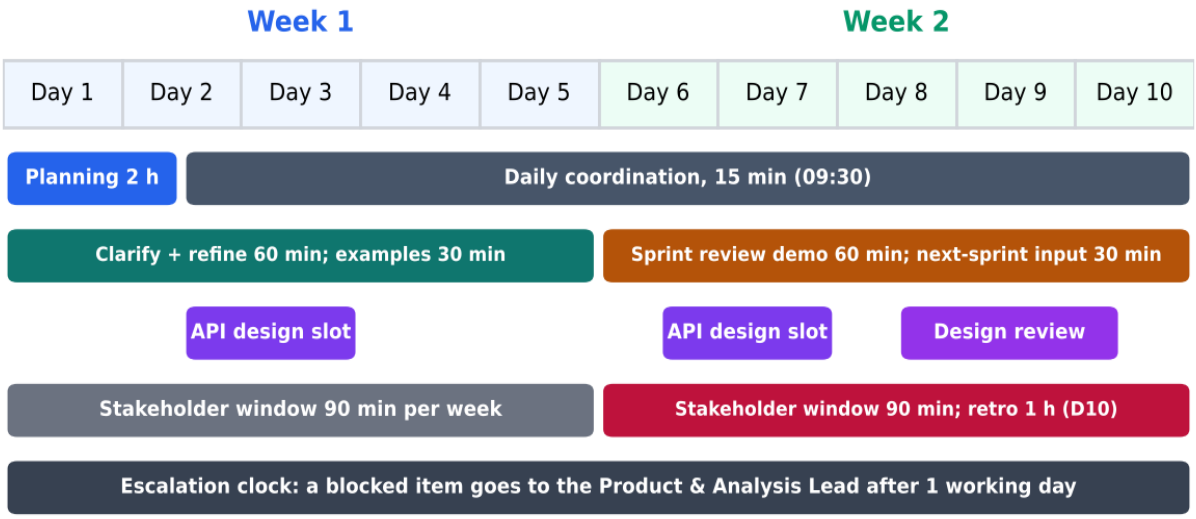


Figure 2. Sprint rhythm for each two-week sprint, including the 90-minute weekly stakeholder window (planned cadence).

Table 12. Cadence and stakeholder participation. Stakeholders attend only the 90-minute weekly window.

Event	When / length	Who	Output
Release-level refinement	Before Sprint 1; 2 h	PL, whole team	Ready first sprint, ordered backlog
Sprint planning	Day 1; 2 h	Whole team (PL for scope)	Sprint goal, selected stories within capacity ceiling
Daily coordination	Every day; 15 min	Team	Blocked items, WIP at limit, help needed
Clarification and refinement	Week 1; 60 min + 30 min examples	PL, warehouse and finance stakeholders, relevant engineers	Acceptance examples (including a negative case) for next stories
API design slot	Twice per sprint; 30 min	API, INV, PSE, FE	Contract changes agreed before coding
Sprint review	Week 2; 60 min	Team, warehouse, finance, support	Working increment accepted or rejected
Retrospective	Last day; 1 h	Team only	One flow improvement carried into next sprint

Table 13. Escalation procedure.

Trigger	Escalate to	By when	Expected response
A story is blocked for one working day	Product & Analysis Lead	Next daily meeting	Unblock by decision, pairing or re-sequencing
Blocked for two working days or sprint goal at risk	PL with project sponsor	Same day	Scope negotiation; defer lower-priority story
Stakeholder unavailable for a needed decision	PL decides on a documented assumption	Within the sprint	Assumption validated at the next review
Defect in authorisation, refund approval or payment	Whole team swarm; PL informed	Immediately	Release gate: feature cannot be Done until fixed
Gateway access further delayed	PL and PSE	On notice	Keep local adapter; replan integration story

2.4 Lean practices applied

Table 14. Lean principles mapped to controls and measures.

Waste	Where it would appear	Practice adopted	Evidence it works
Waiting	Gateway access; review and QA queue; stakeholder decisions	Payment stub and contract first; WIP 2 on review and test; 90-minute clarification window	Blocked story-days; test-queue wait
Unnecessary handovers	Order to payment to inventory	Cross-role stories with one owner and one board; API design slot; swarm when idle	Number of status changes per story; handover comments
Rework	Late discovery of missing cases	Acceptance examples with a negative case before work starts (Definition of Ready); TDD for rules	Reopened stories per completed story
Excess WIP	Starting everything on day 1	WIP limits 4 / 2 / 2; stop starting, start finishing	WIP chart; cycle time
Overproduction	Building analytics before records are trustworthy	Defer OFP-18/19; Must-first ordering	Scope of deferred items
Partially done work	Half-finished stories carried over	Unfinished stories return to the backlog; never counted as partly done	Carryover points

2.5 Project-management tool investigation and selection

I investigated four tools against the criteria in the brief: planning, workflow management, collaboration, reporting, accessibility and cost. Plan limits and prices change often, so each statement below is marked for verification before use.

Table 15. Tool investigation (statements about plans must be verified at the time of use).

Criterion	Jira Software	GitHub Projects	Azure DevOps Boards	Trello
Planning	Epics, backlog, sprints, story estimates, links	Issues in table, board and roadmap views; iteration fields	Work items, backlogs, sprints, delivery plans	Cards and lists; no native sprint or estimate model
Workflow management	Custom statuses and transitions; column constraints (WIP); flags for blocked	Board columns and automation; check support for WIP limits	Custom states; board column limits	Lists; limits via Power-Ups
Collaboration	Comments, mentions, filters, quick filters per team	Strong with pull requests and code review	Strong with repos and pipelines	Easy sharing; light
Reporting	Sprint report, burndown, velocity, cumulative flow (check plan)	Insights charts; basic	Velocity, burndown, CFD	Limited without add-ons
Accessibility	Keyboard navigation and screen-reader support; test with my reviewers	Browser-based, good keyboard support	Browser-based, keyboard support	Keyboard support; some drag-only interactions
Cost (verify)	Free tier for small teams (up to 10 users at time of writing), paid plans per user	Included with GitHub accounts	Free for the first 5 users, paid thereafter	Free tier with limits; paid for advanced views

Table 16. Weighted tool comparison (my judgement).

Criterion	Weight %	Jira	GitHub	Azure	Trello
Planning	20	5	3	4	2
Workflow and WIP	20	5	3	4	2
Collaboration	10	4	4	4	4
Reporting	15	4	2	4	1
Accessibility	10	4	4	4	4
Cost	15	3	5	3	5
Traceability to code	10	4	5	5	2
Weighted score (max 5)	100	**4.25**	**3.55**	**3.95**	**2.70**

Selected: Jira. It scores highest (4.25) because it models epics, sprints, relative estimates, status workflows with column constraints, blocked flags and sprint reports in one place, which is exactly what this strategy needs. It also keeps the single-board rule that removes duplicate status reporting.

Table 17. Tool limitations and mitigations.

Limitation	How I address it
Free plan user cap and some reports depend on the plan	Check the plan before Sprint 1. The seven-member team fits the free tier; the second team in Task 6 does not, so a paid plan (or moving read-only stakeholders to view access) is a costed decision.
Jira statuses can be confused with order states	Separate development workflow from operational order lifecycle (Figure 7); prefix order states in the product, never in Jira.
Reports I need may be missing on a plan	Export issues to a spreadsheet and compute completion, cycle time and blocked days there; this report does so.
Jira can encourage ceremony over flow	Keep one board, a minimal workflow and WIP limits; remove fields nobody uses.
Accessibility of drag-and-drop	Use keyboard transitions and the issue dialog for status changes; ask a reviewer to test with a screen reader.
Personal data in tickets	Use synthetic data and ticket references only; no customer names or card numbers in Jira.

Task 3 – Construct a Prioritised Product Backlog in Jira (8 marks)

3.1 Project set-up in Jira

Table 18. Jira project configuration (described; the CSV import in Appendix B creates the backlog).

Setting	Choice
Project	Order Fulfilment Platform, key OFP, Scrum template
Issue types	Epic, Story, Task, Bug, Spike
Custom fields	Story point estimate; Priority (MoSCoW label); Owner role; Depends on (issue links "is blocked by"); Release gate (yes/no)
Components and labels	Components: Order, Stock, Payments, Fulfilment, Support, Platform. Labels: simulated, privacy, accessibility, region
Board	One board; columns Ready, In Progress, In Review, In Test, Done; column constraints 4 / 2 / 2; Blocked as a flag with reason and owner
Quick filters	Blocked items; Must only; By owner role; Cross-team (added in Task 6)
Versions	Release 1.0 (festival sale); sprints Sprint 1 to Sprint 4

3.2 Epics and the release decision

Table 19. Epics (E6 Regional Operations is added in Task 6).

Epic	Purpose	Stories	Release decision
E1 Order & Stock Core	Place an order and hold stock safely	OFP-1, 2, 3, 4, 14	In
E2 Payments & Refund Governance	Safe payment and audited refund	OFP-5, 6, 7, 8, 9	In
E3 Warehouse Fulfilment & Tracking	Pick, dispatch and show status	OFP-10, 11, 12, 13	In
E4 Support & Reproducible Delivery	Support visibility and repeatable build/tests	OFP-15, 16, 17	In
E5 Insights & Promotions	Analytics, forecasting, promotions	OFP-18, 19	Deferred (roadmap)

3.3 Prioritisation method

I use **MoSCoW for scope** and a **simple weighted score for order inside a priority**. Each story gets Business value (BV), Journey criticality (JC: does the order-to-refund journey break without it?) and Risk or time sensitivity (RR: gateway, concurrency, unknowns), each 1 to 5, divided by size in points: **score = (BV + JC + RR) / points**. A story is a Must if JC is 4 or 5 (the journey breaks without it), a Should if it improves the journey and a Could/Won't if it does not affect it. The score is an input, not a law: dependencies override it. OFP-4 scores 3.0, lower than OFP-2, but it must come first because the payment, pick list and dispatch stories depend on reservation.

Table 20. Prioritisation scores (my judgement from the simulated stakeholder evidence).

ID	Story	BV	JC	RR	Pts	Score	MoSCoW
OFP-1	Role-based sign-in	5	5	3	3	4.3	Must
OFP-2	Seeded catalogue and cart	4	4	1	2	4.5	Must
OFP-3	Checkout creates a persisted order	5	5	3	3	4.3	Must
OFP-4	Reserve stock at checkout with expiry	5	5	5	5	3.0	Must
OFP-5	Idempotent payment request (local stub)	5	5	5	3	5.0	Must
OFP-6	Gateway sandbox integration and confirmation	5	5	5	5	3.0	Must
OFP-7	Return request (owner only, window rule)	4	4	2	3	3.3	Must

ID	Story	BV	JC	RR	Pts	Score	MoSCoW
OFP-8	Refund approval with role/amount limit and audit	5	5	4	5	2.8	Must
OFP-9	Payment reconciliation export	4	3	3	3	3.3	Should
OFP-10	Pick list for reserved and paid orders	5	5	3	3	4.3	Must
OFP-11	Dispatch and audited status transitions	5	5	3	5	2.6	Must
OFP-12	Customer tracking page (accessible)	4	4	2	3	3.3	Must
OFP-13	Barcode scan pick confirmation	3	2	3	3	2.7	Should
OFP-14	Low-stock alert (fixed threshold rule)	3	2	1	2	3.0	Should
OFP-15	Repeatable build, deploy and smoke checks	4	4	4	5	2.4	Must
OFP-16	Support console: order and refund lookup	4	4	2	3	3.3	Must
OFP-17	End-to-end journey test and seed data	3	4	4	3	3.7	Must
OFP-18	Sales analytics dashboard	2	1	1	5	0.8	Could
OFP-19	Demand forecasting (ML, proposed extension)	2	1	3	8	0.8	Won't

3.4 Backlog: stories, estimates, dependencies and owners

Estimates use relative story points (Task 4 explains the method). OFP-5 originally listed only OFP-3 as its dependency; the Sprint 1 simulation added OFP-4 (Task 5), shown below as the corrected dependency.

Table 21. Product backlog. *Dependency added after Sprint 1 simulation. Sprint = original plan (before the Day 4 change).

ID	Story	Epic	Priority	Pts	Depends on	Owner role	Sprint
OFP-1	Role-based sign-in	E1	Must	3	None	PSE	S1
OFP-2	Seeded catalogue and cart	E1	Must	2	OFP-3	FE	S2
OFP-3	Checkout creates a persisted order	E1	Must	3	OFP-1	API	S1
OFP-4	Reserve stock at checkout with expiry	E1	Must	5	OFP-3	INV	S1
OFP-5	Idempotent payment request (local stub)	E2	Must	3	OFP-3, OFP-4*	PSE	S1
OFP-6	Gateway sandbox integration and confirmation	E2	Must	5	OFP-5 + gateway (wk 4)	PSE	S3
OFP-7	Return request (owner only, window rule)	E2	Must	3	OFP-11	API	S3
OFP-8	Refund approval with role/amount limit and audit	E2	Must	5	OFP-6, OFP-7	PSE	S4
OFP-9	Payment reconciliation export	E2	Should	3	OFP-6	INV	S4
OFP-10	Pick list for reserved and paid orders	E3	Must	3	OFP-4, OFP-5	INV	S2
OFP-11	Dispatch and audited status transitions	E3	Must	5	OFP-10	API	S2
OFP-12	Customer tracking page (accessible)	E3	Must	3	OFP-11	FE	S3
OFP-13	Barcode scan pick confirmation	E3	Should	3	OFP-10 + scanner (wk 5)	INV	S3
OFP-14	Low-stock alert (fixed threshold rule)	E1	Should	2	OFP-4	INV	S2
OFP-15	Repeatable build, deploy and smoke checks	E4	Must	5	OFP-1	DEVOPS	S1
OFP-16	Support console: order and refund lookup	E4	Must	3	OFP-7, OFP-11	FE	S4
OFP-17	End-to-end journey test and seed data	E4	Must	3	OFP-8, OFP-11	QA	S4
OFP-18	Sales analytics dashboard	E5	Could	5	OFP-11	FE	Deferred
OFP-19	Demand forecasting (ML, proposed extension)	E5	Won't	8	OFP-18	INV	Deferred

SIMULATED / ILLUSTRATIVE

Jira-style mock-up | Order Fulfilment Platform (OFP)

Backlog - Issues

Project OFP | list view | all stories (illustrative mock-up)

Key	Summary	Epic	Priority	Pts	Sprint	Owner
OFP-1	Role-based sign-in	E1	Must	3	Sprint 1	PSE
OFP-2	Seeded catalogue and cart	E1	Must	2	Sprint 2	FE
OFP-3	Checkout creates a persisted order	E1	Must	3	Sprint 1	API
OFP-4	Reserve stock at checkout with expiry	E1	Must	5	Sprint 1	INV
OFP-5	Idempotent payment request (local stub)	E2	Must	3	Sprint 1	PSE
OFP-6	Gateway sandbox integration and confirmation	E2	Must	5	Sprint 3	PSE
OFP-7	Return request (owner only, window rule)	E2	Must	3	Sprint 3	API
OFP-8	Refund approval with role/amount limit an...	E2	Must	5	Sprint 4	PSE
OFP-9	Payment reconciliation export	E2	Should	3	Sprint 4	INV
OFP-10	Pick list for reserved and paid orders	E3	Must	3	Sprint 2	INV
OFP-11	Dispatch and audited status transitions	E3	Must	5	Sprint 2	API
OFP-12	Customer tracking page (accessible)	E3	Must	3	Sprint 3	FE
OFP-13	Barcode scan pick confirmation	E3	Should	3	Sprint 3	INV
OFP-14	Low-stock alert (fixed threshold rule)	E1	Should	2	Sprint 2	INV
OFP-15	Repeatable build, deploy and smoke checks	E4	Must	5	Sprint 1	DEVOPS
OFP-16	Support console: order and refund lookup	E4	Must	3	Sprint 4	FE
OFP-17	End-to-end journey test and seed data	E4	Must	3	Sprint 4	QA
OFP-18	Sales analytics dashboard	E5	Could	5	Backlog	FE
OFP-19	Demand forecasting (ML, proposed extension)	E5	Won't	8	Backlog	INV

Figure 3. Jira-style backlog list for project OFP (illustrative mock-up).

SIMULATED / ILLUSTRATIVE

Jira-style mock-up | Order Fulfilment Platform (OFP)

Backlog - Sprint planning

Four sprints plus deferred work (illustrative mock-up)

Sprint 1 (weeks 1-2) | 16 points			
OFP-1	Role-based sign-in	Must	3
OFP-3	Checkout creates a persisted order	Must	3
OFP-4	Reserve stock at checkout with expiry	Must	5
OFP-5	Idempotent payment request (local stub)	Must	3
OFP-15a	Repeatable local build and smoke check (slice a)	Must	2
Sprint 2 (weeks 3-4) | 15 points			
OFP-2	Seeded catalogue and cart	Must	2
OFP-10	Pick list for reserved and paid orders	Must	3
OFP-11	Dispatch and audited status transitions	Must	5
OFP-14	Low-stock alert (fixed threshold rule)	Should	2
OFP-15b	CI pipeline and deployment notes (slice b)	Must	3
Sprint 3 (weeks 5-6) | 14 points			
OFP-6	Gateway sandbox integration and confirmation	Must	5
OFP-7	Return request (owner only, window rule)	Must	3
OFP-12	Customer tracking page (accessible)	Must	3
OFP-13	Barcode scan pick confirmation	Should	3
Sprint 4 (weeks 7-8) | 14 points			
OFP-8	Refund approval with role/amount limit and audit	Must	5
OFP-9	Payment reconciliation export	Should	3
OFP-16	Support console: order and refund lookup	Must	3
OFP-17	End-to-end journey test and seed data	Must	3
Deferred / not scheduled | Epic E5 Insights & Promotions			
OFP-18	Sales analytics dashboard	Could	5
OFP-19	Demand forecasting (ML, proposed extension)	Won't	8

Figure 4. Jira-style backlog by sprint, with deferred epic E5 (illustrative mock-up).

3.5 Acceptance criteria

Table 22. Acceptance criteria. Every user-facing or access-controlled story includes at least one negative case.

Story	Acceptance criteria
OFP-1 Role-based sign-in	AC1 A valid sign-in returns only the permissions of that role. AC2 Expired or tampered session is rejected (401) and returns no data (negative case). AC3 A picker calling the refund-approval endpoint gets 403.
OFP-2 Seeded catalogue and cart	AC1 Cart total equals the sum of seeded prices x quantity. AC2 An item cannot be added beyond available stock. AC3 Cart is operable by keyboard with labelled fields.
OFP-3 Checkout creates a persisted order	AC1 A valid cart creates exactly one order with owner, validated address and status Placed, persisted after restart. AC2 Empty cart or missing address creates no order (negative case). AC3 Creation is written to the audit record.
OFP-4 Reserve stock at checkout with expiry	AC1 Checkout reserves the requested quantity with a 15-minute expiry. AC2 100 concurrent orders for a 10-unit item give at most 10 reservations and never a negative count. AC3 Expired or cancelled reservations return stock to available.
OFP-5 Idempotent payment request (local stub)	AC1 Ten retries with the same idempotency key create one payment intent and one order. AC2 Stub failure leaves the order recoverable and retry-safe. AC3 No card number is stored or logged; synthetic tokens only.

Story	Acceptance criteria
OFP-6 Gateway sandbox integration and confirmation	AC1 Sandbox success marks the order Paid once; a duplicate callback is ignored. AC2 A 10-second timeout leaves the order pending and retries with back-off, never charging twice. AC3 Each payment id maps to one order id.
OFP-7 Return request (owner only, window rule)	AC1 Only the order owner can request a return, for a Delivered order inside 14 days. AC2 Other users or late requests are rejected with a reason (negative case). AC3 Support can see the request status.
OFP-8 Refund approval with role/amount limit and audit	AC1 A finance reviewer approves or rejects; a support agent attempting approval gets 403. AC2 Refunds above INR 10,000 need a second approver. AC3 Each decision logs who, when, amount and reason; a refund cannot exceed the amount paid.
OFP-9 Payment reconciliation export	AC1 Export lists payments with matching order, amount and status for a date range. AC2 Mismatches are flagged for human review.
OFP-10 Pick list for reserved and paid orders	AC1 A picker sees only reserved and paid orders for their warehouse. AC2 The list shows item, bin, quantity and reservation expiry. AC3 An unpaid order never appears (negative case).
OFP-11 Dispatch and audited status transitions	AC1 Only valid transitions (Paid, Picked, Dispatched, Delivered) are accepted; Placed to Delivered is rejected. AC2 Each transition records actor and time. AC3 Dispatching the same order twice creates one dispatch.
OFP-12 Customer tracking page (accessible)	AC1 A customer sees only their own order and permitted milestones, without internal notes. AC2 Status is conveyed by text, not colour alone; screen-reader labels present. AC3 Page is fully keyboard operable.
OFP-13 Barcode scan pick confirmation	AC1 A valid scan confirms the item; a wrong item is rejected with a message. AC2 A manual override needs a reason and is audited.
OFP-14 Low-stock alert (fixed threshold rule)	AC1 When available stock is at or below the fixed threshold, exactly one alert is raised until restock. AC2 The rule is a configurable fixed threshold; no prediction is claimed.
OFP-15 Repeatable build, deploy and smoke checks	AC1 A clean checkout and the documented script build and start the app; health check passes. AC2 Records persist after restart. AC3 Deployment notes are updated.
OFP-16 Support console: order and refund lookup	AC1 An agent finds an order by number or email and sees return and refund status. AC2 Personal data are masked except what the task needs. AC3 Agents cannot edit payment data (negative case).
OFP-17 End-to-end journey test and seed data	AC1 The script drives one order to Delivered and one return to Refunded on synthetic data, with an audit record per transition. AC2 An unauthorised refund attempt is rejected inside the script (negative case).
OFP-18 Sales analytics dashboard	AC1 Totals reconcile with order records. (Deferred.)
OFP-19 Demand forecasting (ML, proposed extension)	AC1 Forecast is labelled a model, with evaluation against held-out data. (Proposed extension, deferred.)

3.6 Definition of Ready and Definition of Done

Table 23. Definitions of Ready and Done. UI completion alone is never sufficient.

Definition of Ready (before a story enters a sprint)	Definition of Done (before a story is Done)
1. User, outcome and priority understood 2. Acceptance examples agreed, including a negative case 3. Dependencies identified and contract or stub available 4. Privacy questions resolved (what personal data, who sees it) 5. Accessibility needs noted for user-facing work 6. Sized at 5 points or less; larger stories are split 7. Fits one sprint and the capacity ceiling	1. Code reviewed by a second engineer 2. Relevant automated tests pass; new code covered by tests (target 70%, assumption) 3. Persistent state and server-side authorisation verified 4. Privacy: no real card or customer data in code, logs, tests or tickets; personal data masked on screen 5. Accessibility: keyboard use, readable labels and non-colour status checked for user-facing screens 6. Audit record present for state changes 7. Deployment notes updated; build reproducible 8. QA accepted against the acceptance criteria; evidence linked in Jira

3.7 Decisions where stakeholder requests competed

Table 24. Prioritisation decisions resolving competing stakeholder requests.

Decision	Competing requests	Choice and reason
1. Stock reservation before management charts	Management wanted an analytics dashboard and all nine modules; pickers wanted reservation and accurate pick lists.	Reservation first (Must) and analytics deferred. Overselling creates failed orders and refunds that no dashboard can repair.
2. Minimal refund approval before promotions or a rich reconciliation dashboard	Support wanted fast refunds; finance wanted full reconciliation and audit; marketing wanted promotions.	A single approval workflow with role and amount limits and an audit record (OFP-8, Must). Reconciliation export is a Should (OFP-9). Promotions deferred. A quick but unauthorised refund is a direct financial loss.
3. Manual pick confirmation vs barcode scanning	Pickers wanted scanning; the scanner device arrives in week 5.	Pick list and dispatch first (Sprint 2); scan confirmation is a Should in Sprint 3 and its fallback is manual confirmation with a reason.

Task 4 – Capacity-Based Release Plan and Delivery Workflow (7 marks)

4.1 Capacity assumptions

Table 25. Capacity assumptions.

Assumption	Value
Team	7 people (roles in Section 2.3), all full time for the 8 weeks
Working days per sprint	10
Focused hours per person per day	6 (meetings, email and context switching already excluded)
Nominal focused hours per sprint	7 x 10 x 6 = 420 h
Reserve for reviews, unplanned defects and integration	20% = 84 h
Focused planning hours per sprint	336 h
Provisional first-sprint ceiling	16 points (assumption, revised after Sprint 1)

Estimation method. Stories are sized with relative story points (1, 2, 3, 5, 8) by a planning-poker round of the whole team. OFP-1 (role-based sign-in, well understood) is the reference story at 3. A story's size reflects effort, complexity and uncertainty relative to the reference, not elapsed time. Anything of 5 points or more is checked for splitting, and 8 must be split. Points are **not** converted to hours. As an independent sanity check only, the team lists tasks and hour estimates for the stories selected for a sprint and confirms the total stays below the 336 focused hours (next table); that check cannot change a story's points.

Table 26. Bottom-up sanity check of the selected stories (planning assumption).

Sprint	Points selected	Bottom-up task hours (check only)	Within 336 h?
1	16	298	Yes (89%)
2	15	276	Yes (82%)
3	14	255	Yes (76%)
4	14	262	Yes (78%)

The first selection (OFP-1, 2, 3, 4, 5 and 15) came to 21 points, five above the provisional ceiling. I negotiated it down: the cart screen (OFP-2, 2 points) moves to Sprint 2 because checkout can be driven by an API with seeded product ids; OFP-15 is split into a 2-point local build and smoke check (OFP-15a) kept in Sprint 1, and a 3-point CI pipeline and deployment notes (OFP-15b) moved to Sprint 2. The result is 16 points. Sprint 4 is deliberately light for carryover, regression and a release rehearsal.

4.2 Sprint goals and selected work

Table 27. Release plan: four sprints (original plan before the Day 4 change). Total 59 points.

Sprint	Goal	Stories	Pts
1 (weeks 1-2)	Secure ordering foundation: a signed-in customer places a persisted order that reserves stock safely; payment requests are retry-safe (stub); the build is repeatable.	OFP-1, OFP-3, OFP-4, OFP-5, OFP-15a	16
2 (weeks 3-4)	Fulfilment core: a paid order can be picked and dispatched with audited status; cart UI and deployable test environment.	OFP-2, OFP-10, OFP-11, OFP-14, OFP-15b	15
3 (weeks 5-6)	Money and returns: gateway sandbox payment confirmation, customer tracking, return request and barcode pick confirmation.	OFP-6, OFP-7, OFP-12, OFP-13	14
4 (weeks 7-8)	Controlled refund and support visibility: refund approval with audit, support lookup, reconciliation, hardening and release rehearsal.	OFP-8, OFP-9, OFP-16, OFP-17	14

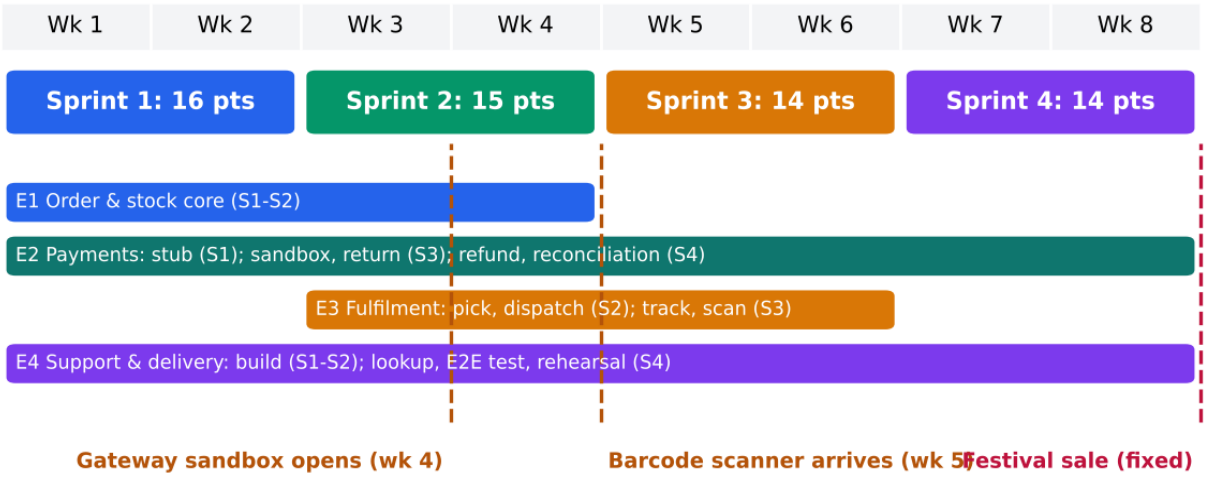
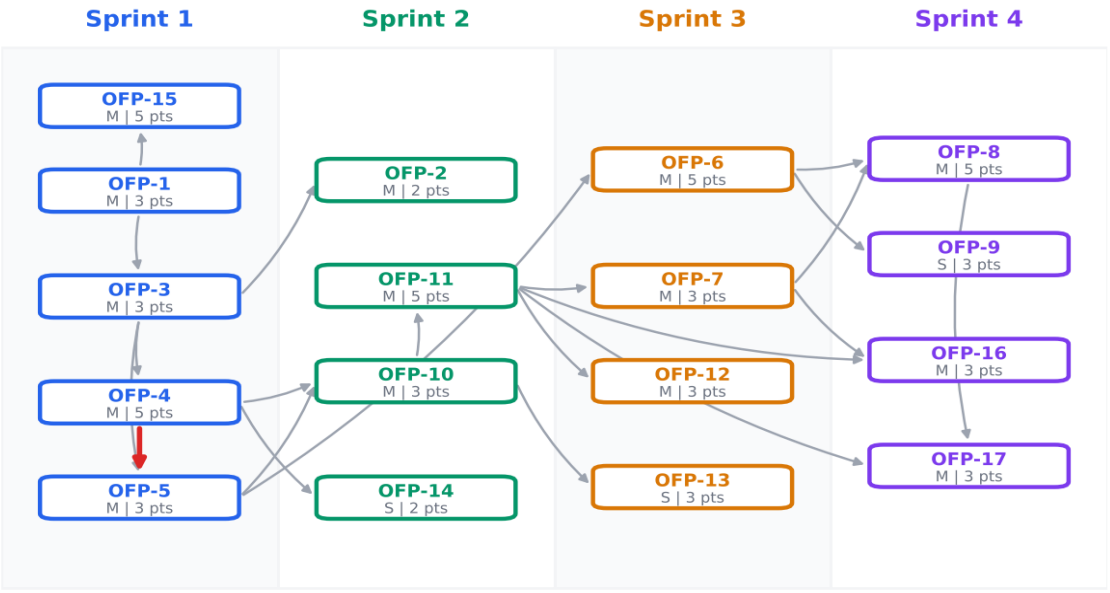


Figure 5. Eight-week release timeline with the gateway (week 4), scanner (week 5) and festival constraints.

4.3 Dependencies and constraints

- **Gateway sandbox opens in week 4.** Sprint 1 and 2 use a payment stub behind a fixed contract (OFP-5). The real integration (OFP-6) is planned for Sprint 3 so that the stub, not the gateway, absorbs the delay. If access slips again, the replay tests keep OFP-6 from blocking OFP-7 and OFP-8.
- **Scanner arrives in week 5.** Scan confirmation (OFP-13) sits in Sprint 3; manual confirmation with a reason is the fallback.
- **Cross-functional responsibility.** Each story has one owner role but needs another: OFP-5 needs INV for the reservation result; OFP-11 needs API plus INV; OFP-12 and OFP-16 need FE plus API; QA tests every story.
- **Corrected after simulation.** OFP-5 depends on OFP-4 (payment is requested only for a reserved order).



Gateway sandbox opens in week 4 -> OFP-6 (gateway integration) waits for Sprint 3.
Barcode scanner arrives in week 5 -> OFP-13 (scan confirmation) sits in Sprint 3.
Red arrow = dependency added after the Sprint 1 simulation (OFP-4 -> OFP-5). Arrows point from prerequisite to dependent story.

Figure 6. Story dependency network by sprint (prerequisite to dependent). The red arrow is the dependency found in the Sprint 1 simulation.

4.4 Development workflow in Jira and its separation from the order lifecycle

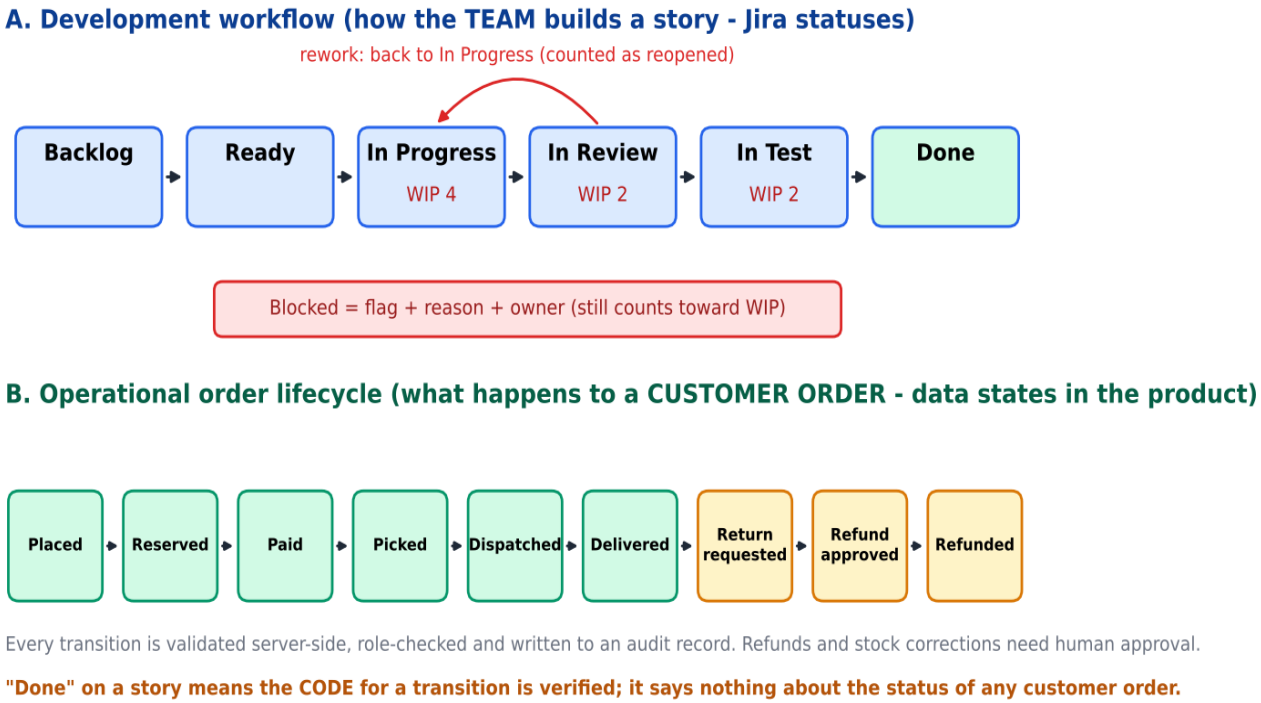


Figure 7. Development workflow (team statuses in Jira) versus the operational order lifecycle (product data states).

Table 28. Workflow transition rules.

Transition	Rule (must be true to move)	Completion check
Backlog to Ready	Definition of Ready met	PL confirms
Ready to In Progress	Owner assigned; dev WIP below 4; dependencies available or stubbed	Owner pulls the story
In Progress to In Review	Code complete; unit tests pass; acceptance evidence attached	Pull request open
In Review to In Test	Second engineer approved; privacy and accessibility items checked; test WIP below 2	Review checklist
In Test to Done	QA ran the acceptance criteria including the negative case; Definition of Done met	QA signs off with evidence
Any to In Progress (reopen)	Failed review or test; counted as reopened	Reason in comment
Any to Blocked (flag)	Reason, owner and blocker link recorded; item still counts in its column	Escalate after 1 working day

Why two lifecycles. A Jira story reaching Done means the code for a transition (for example Paid to Picked) is verified. It says nothing about the status of any customer order, which is a data state in the product with its own validation, permissions and audit record. Keeping the two apart prevents Jira from being used as an order tracker and prevents "Done" being read as "shipped".

4.5 Work-in-progress limits and flow policies

Table 29. WIP limits and why.

Column	Limit	Justification
In Progress (includes Blocked)	4	Seven people, with QA, DevOps and PL not coding all the time: about one item per two engineers forces pairing and finishing. Blocked items stay counted so blocked work cannot hide.
In Review	2	Reviews are quick; a longer queue means code is waiting, not being improved.
In Test	2	There is one QA engineer, so this is the tightest constraint. A third item would only wait.

Table 30. Blocked, urgent and unfinished work.

Situation	Policy
Blocked work	Flag with reason, owner and blocker link. Escalate to the PL after one working day. The owner pulls another item only if the WIP limit allows; otherwise they help unblock or review.
Urgent request	One expedite item may enter. The PL removes planned work of equal size first, and the change is visible on the board. Security and refund defects always qualify.
Unfinished stories at sprint end	Return to the ordered backlog; never marked partly done and not counted as completed velocity. Carryover is shown separately next sprint.
Idle capacity	Stop starting, start finishing: help review or test before pulling new work.

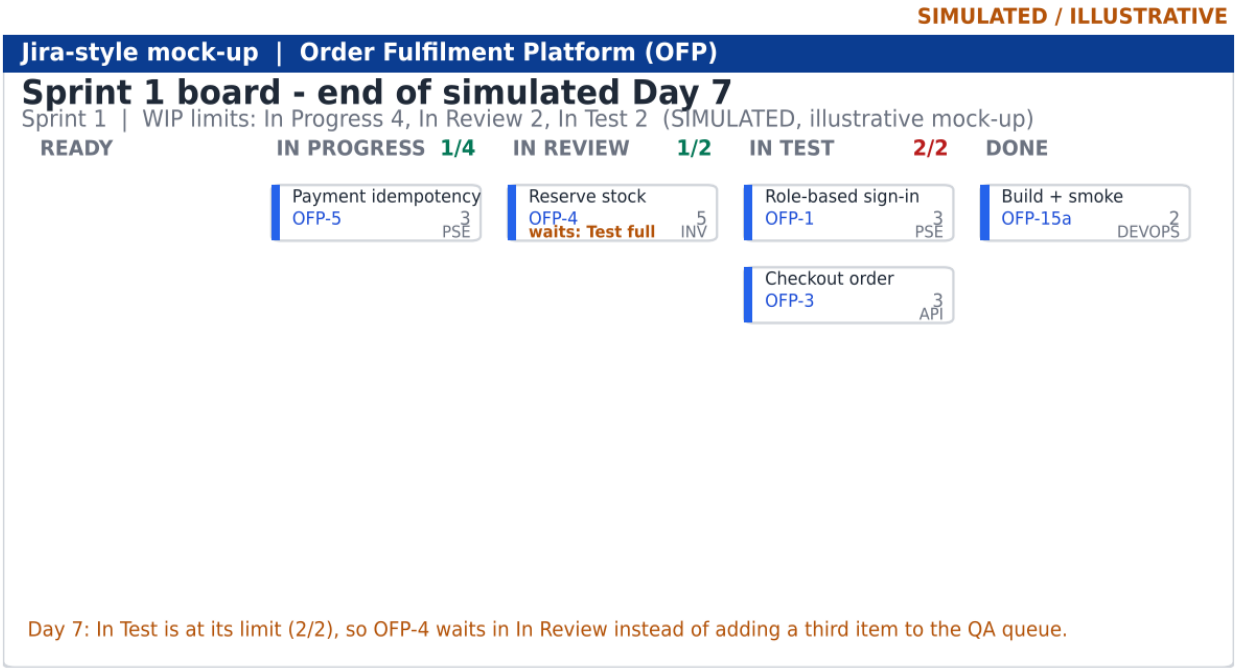


Figure 8. Sprint board with WIP limits, shown at the end of simulated Day 7 of Sprint 1 (SIMULATED, illustrative mock-up; the event log is in Task 5).

4.6 How the selected features achieve the product goal

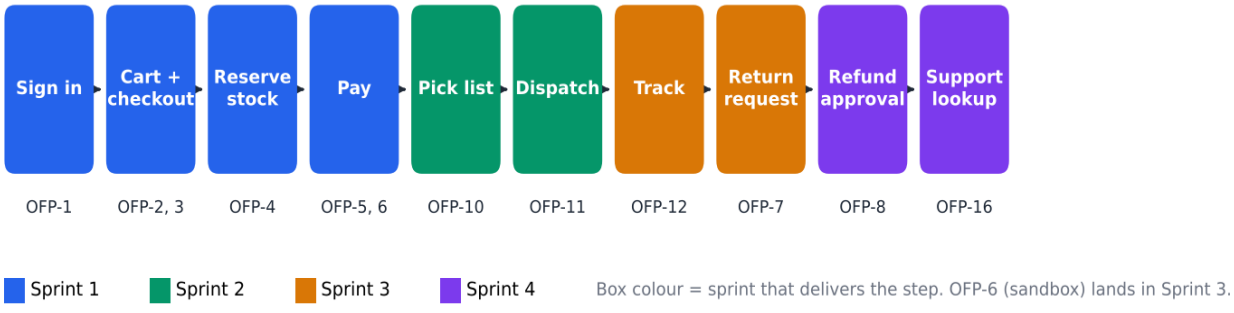


Figure 9. Selected stories form one order-to-delivery-to-refund journey; box colour shows the delivering sprint.

Every step of the journey in Figure 9 has a Must story. Without OFP-4 stock cannot be trusted; without OFP-5 and OFP-6 an order cannot be marked Paid; without OFP-10 and OFP-11 nothing can be picked or dispatched; without OFP-12 customers cannot see status; and without OFP-7 and OFP-8 no return can reach an authorised refund. The Should items (OFP-9, 13, 14) and the deferred analytics epic improve the journey but are not required for it to work, so they are the first to move if capacity falls short.

Task 5 – Simulate Delivery and Validate the Design (7 marks)

Label: Everything in this task is SIMULATED. It is a planning rehearsal constructed from my own plan and is not a record of actual team activity. The Jira-style figures are mock-ups.

5.1 Prediction before the simulation

Predicted bottleneck: the QA test queue and the hand-off from order and reservation to payment. Stock reservation (OFP-4), the persisted order (OFP-3) and the payment key (OFP-5) converge on the same data and the same single QA engineer, so I expected work to pile up in In Test and the payment story to wait for the order slice.

Table 31. Prediction: rules that should control the bottleneck and the evidence that would show they work.

Control rule	Why it should help	Evidence that it is effective
WIP limit 2 on In Test and In Review	Limits the QA queue and exposes waiting early	Test queue never exceeds 2; queue wait of at most 1 story-day; item waits visibly in Review rather than invisibly in Test
Payment stub and contract before gateway (OFP-5)	Removes dependence on week-4 sandbox access	OFP-5 can start without gateway access
Escalation after one blocked working day	Shortens blocked time	Blocked item resolved within one day of escalation
Definition of Ready with a negative case	Reduces rework	Reopened stories per completed story at most 0.25

5.2 Event log and Jira updates (SIMULATED)

Table 32. Sprint 1 event log (SIMULATED).

#	Day	Simulated event	Jira update	Effect
1	D1	Sprint planning commits 16 points: OFP-1, 3, 4, 5, 15a. OFP-1, 3 and 15a are pulled.	Sprint 1 started; three stories to In Progress	Dev WIP 3 of 4
2	D2	Clarification slot with the warehouse stakeholder fixes reservation expiry at 15 minutes and release on cancellation; acceptance examples for OFP-4 agreed. OFP-4 pulled.	OFP-4 to Ready then In Progress; decision recorded as a comment	Dev WIP 4 of 4; OFP-4 started one day late because it was not Ready on day 1
3	D3	OFP-5 cannot be pulled: dev WIP is at the limit and its owner (PSE) is on OFP-1.	OFP-5 stays Ready with a "waiting for owner" comment	Waiting made visible
4	D4	OFP-1 and OFP-15a finish development.	Both to In Review	Review 2 of 2
5	D5	Review finds no negative test for an expired session on OFP-1, so it returns to development. OFP-3 enters Review; OFP-15a enters Test.	OFP-1 back to In Progress with reason (reopened 1)	Reopened 1
6	D6	OFP-15a fails the Definition of Done check (deployment notes missing); notes added the same day. OFP-3 moves to Test; OFP-1 re-enters Review; OFP-4 enters Review. PSE pulls OFP-5.	OFP-15a Done; OFP-5 In Progress	OFP-15a cycle time 6 days
7	D7	OFP-1 enters Test. Test is now 2 of 2 (OFP-3, OFP-1), so OFP-4 waits in Review.	Board shows Test at its limit (Figure 8)	Test-queue wait 1 story-day
8	D8	OFP-1 and OFP-3 are Done. OFP-4 enters Test and fails a concurrency check: two orders reserve the last unit. OFP-5 is flagged Blocked because its confirmation tests need the reservation result.	OFP-4 reopened (2); OFP-5 flagged Blocked; link "is blocked by OFP-4"	Blocked story-day 1
9	D9	Escalation after one blocked day: the PL asks INV and QA to pair; fix is a row-level lock plus a unique constraint. OFP-4 returns to Review. OFP-5 still blocked.	OFP-4 to In Review; escalation comment	Blocked story-day 2
10	D10	OFP-4 passes review and retest and is Done. OFP-5 is unblocked only at day end and cannot finish.	OFP-4 Done; OFP-5 returned to the backlog as carryover (3 points)	Blocked story-day 3

#	Day	Simulated event	Jira update	Effect
11	D10	Sprint review (simulated): stakeholders accept the reservation behaviour and ask for the expiry time to be shown on the pick list.	New acceptance criterion added to OFP-10	None
12	D10	Retrospective; indicators exported from Jira to a spreadsheet and analysed.	Sprint closed; carryover re-ordered	Indicators below

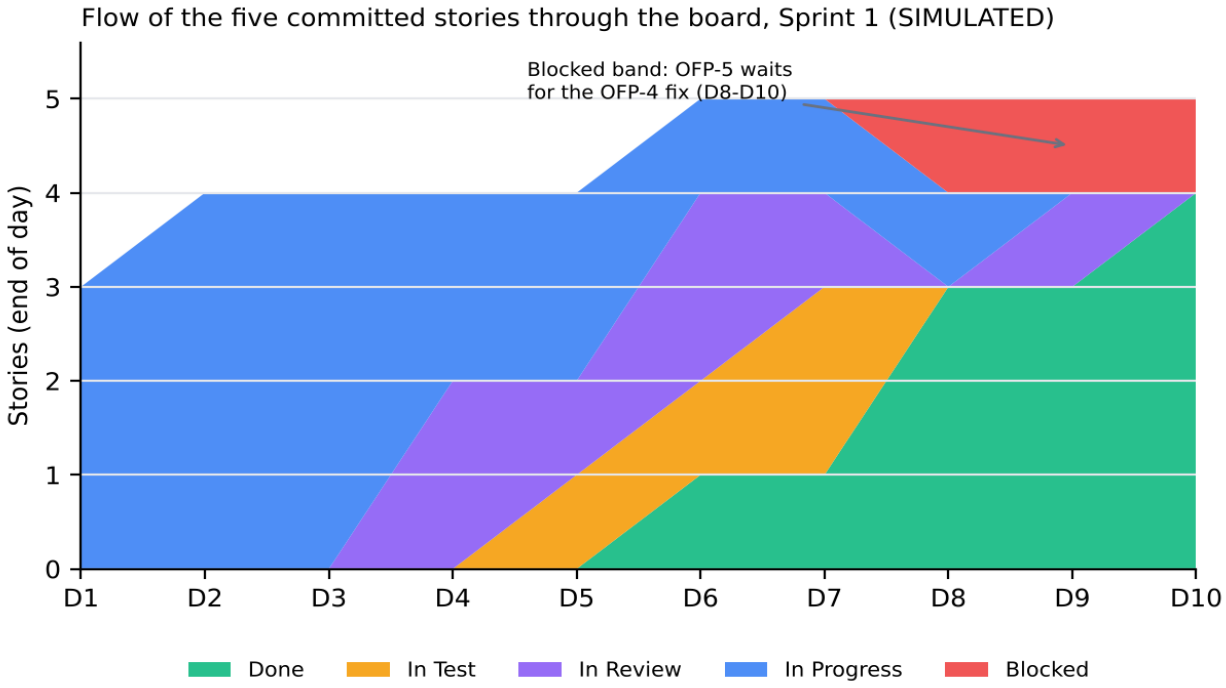


Figure 10. Sprint 1 flow of the five committed stories through the board columns (SIMULATED).

5.3 Delivery indicators (SIMULATED)

Table 33. Sprint 1 indicators (SIMULATED).

Indicator	Result	Target	Status
Completed versus planned points	13 of 16 = 81.3%	At least 80%	Met narrowly
Blocked time	3 story-days (OFP-5, D8 to D10)	At most 2	Missed
Reopened stories per completed story	2 / 4 = 0.50	At most 0.25	Missed
Cycle time (start to Done)	Mean 7.75 days (OFP-1: 8, OFP-3: 8, OFP-15a: 6, OFP-4: 9)	Informational	Long for a 10-day sprint
Test-queue wait	1 story-day; Test at its limit on 1 day (D7)	At most 1 day	Met
Work in progress	Dev WIP reached 4 on D2-D3 and never exceeded 2 after D4	Within 4 / 2 / 2	Within limits

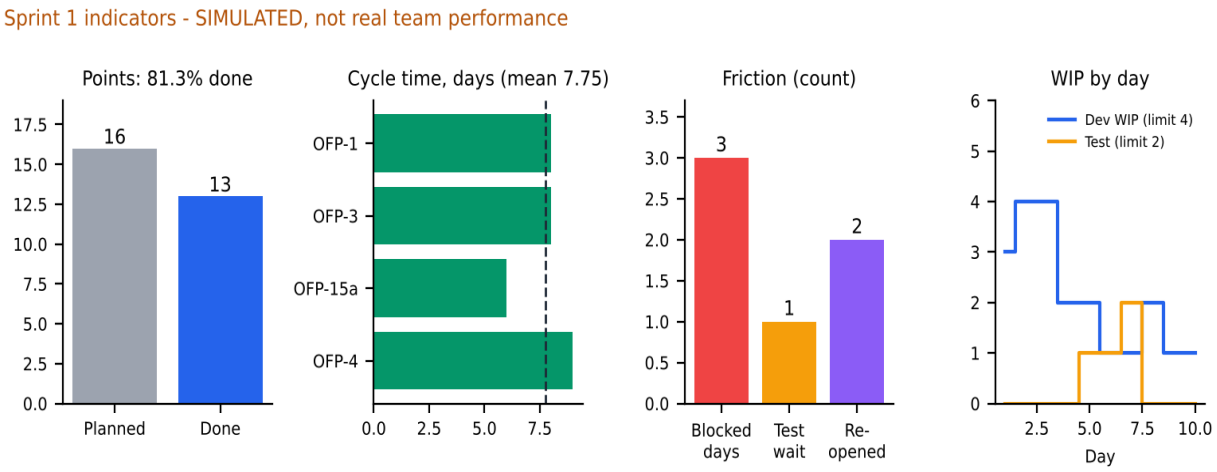


Figure 11. Sprint 1 indicators (SIMULATED).

5.4 Interpretation

- **The prediction was only half right.** The test queue did reach its limit, but it cost one story-day. The larger loss was the three blocked story-days on OFP-5 caused by a dependency I had not recorded: payment is only requested for a reserved order, so OFP-5 depends on OFP-4, not just on OFP-3.
- **The WIP limit made waiting visible but did not prove improvement.** With no baseline sprint, I cannot claim the limit reduced waiting; I can only say that OFP-4 waited visibly in Review instead of silently in Test.
- **Completion of 81.3% hides an unhealthy pattern.** Four stories started within two days and finished between D6 and D10, so testing and rework landed at the end. After D4 the board held at most two active development items while people were available, so the plan underused capacity early and overloaded QA late.
- **The Definition of Ready was not strong enough.** OFP-4 was not truly Ready on D1 (expiry rule open) and its concurrency case was found only in test; OFP-1 lacked a negative case. Both caused reopening.
- **Single-owner risk.** PSE owned OFP-1 and OFP-5, which delayed the start of payments. This foreshadows the staffing change in Task 6.

5.5 Limitation found and improvement for the next sprint

Table 34. Limitation and the resulting improvements.

Limitation in the original plan	Improvement for Sprint 2 (planning or workflow decision)
Missing dependency OFP-4 to OFP-5; parallel start	Record the dependency in Jira; start the producer first and publish the reservation contract stub by day 2 so the consumer can build against it.
Stories not Ready on day 1	Hold the clarification slot in the week before the sprint; Definition of Ready now requires every Must story to have a negative case and concurrency or edge example.
Large 5-point stories finish late and block	Split stories of 5 points or more into slices of 3 or less. OFP-11 (5) becomes OFP-11a status transitions (3) and OFP-11b dispatch screen (2).
Capacity idle early, QA overloaded late	Stagger pulls so that one story reaches Test every two days; add a swarm rule: when dev WIP is below the limit and the test queue is 1 or more, an engineer pairs on review or test before pulling new work.
Single owner for payments	Pair a second engineer on every payments story.

Sprint 2 would carry over OFP-5 (3 points) plus no more than 13 new points under the original plan. The Day 4 changes in Task 6 then reshaped Sprint 2 further.

Task 6 – Redesign Delivery Under Changed Conditions (12 marks)

Label: The changed conditions and every figure below are SIMULATED planning evidence. The revised results are not evidence of real team improvement.

6.A Diagnose the impact (2 marks)

Table 35. Impact analysis, using evidence from the original plan and the Sprint 1 simulation.

Change	Assumptions broken	Capacity	Scope	Dependencies and risks
Two additional regional warehouses	One warehouse, one rule set	Extra analysis and test effort	Region tax, shipping and refund rules; warehouse-scoped stock and pick lists (new stories OFP-20 to 23, 15 points)	New access-control risk: a picker must never see another warehouse; OFP-3, 8, 10, 11 are affected
Second seven-member team joins	Single team, shared context	Newcomers need onboarding; shared APIs need coordination	Work must be split without duplicate code	Integration bottleneck on the order and payment contracts
Payments engineer becomes unavailable	PSE owns OFP-1, 5, 6, 8 (16 of 59 points, 27%)	Original team falls to 6 people	Remaining PSE work: OFP-5 carryover, 6, 8 (11 points) is on the critical path	Loss of payment knowledge; idempotency contract must survive
Region-specific rules	Refund limits and tax are universal	Re-estimation	Policy must be data, not code branches	Rework if rules are hard-coded
Deadline unchanged	Slack existed in Sprint 4	No extra time	Lower-value scope must go	Overcommitment

Evidence from Sprint 1 that matters here. Payments work was already the most fragile (3 blocked story-days on OFP-5 and a single owner), and 13 points were finished per 336 focused hours. Remaining demand after Sprint 1 is 46 points (including the 3-point OFP-5 carryover and the previously planned OFP-13 and OFP-14); the regional work adds 15 points, giving **61 points** to place in three sprints.

Table 36. Recalculated capacity (planning assumptions). Team B availability reflects onboarding: 60%, 90%, 100%.

Sprint	Team A: 6 people	Team B: 7 people x availability	Nominal total	After 20% reserve	After 15% coordination	Effective vs 336 h
2	360 h	252 h (60%)	612 h	490 h	416 h	1.24x
3	360 h	378 h (90%)	738 h	590 h	502 h	1.49x
4	360 h	420 h (100%)	780 h	624 h	530 h	1.58x
Total	1,080 h		2,130 h	1,704 h	1,448 h	1.44x of 1,008 h

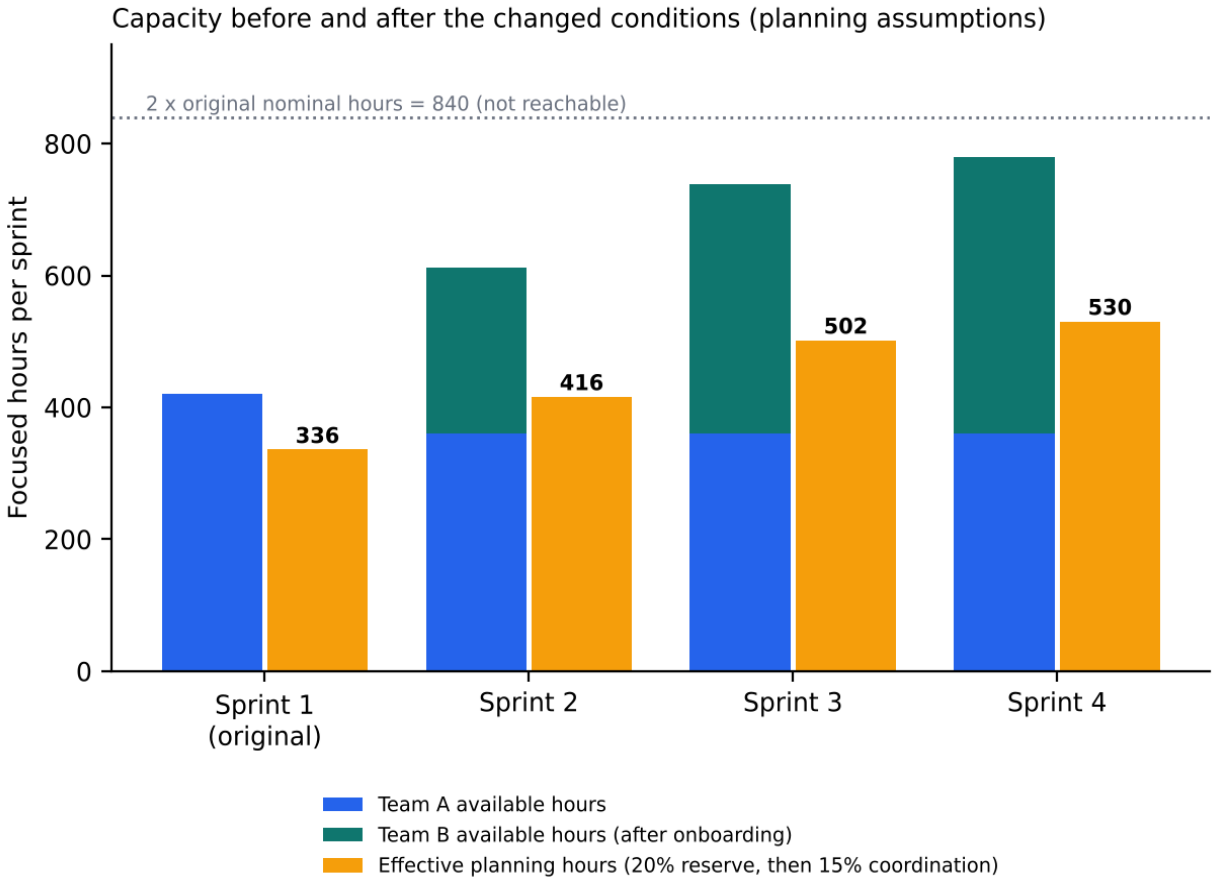


Figure 12. Capacity before and after the changes (assumptions): thirteen people give roughly 1.4x, not 2x, of the original three-sprint effective capacity.

Why not 2x. Newcomers are only partly available while onboarding (40% of Team B's first sprint is reserved for learning), 15% of focused hours go to cross-team coordination, and Team A has lost an engineer. Scaling by capacity ratio from the simulated 13 points per 336 hours gives a forecast of about 16, 19 and 20 points for Sprints 2 to 4 (55 total). A single six-person Team A would manage about 33 points in three sprints, so **doing nothing differently fails the journey** and **adding people without redesign overruns by about 6 points**, which is why scope and coordination must change together.

6.B Measurable redesign objectives (2 marks)

Table 37. Redesign objectives.

ID	Objective	Measure and target
RO1	One shared order and payment contract	Single versioned contract; 0 breaking changes merged without dependency-review sign-off
RO2	Limit cross-team coupling	No more than 2 cross-team dependencies in active implementation at any time (Jira filter)
RO3	Preserve a complete journey	End-to-end demo (order to delivered to refunded) for the central warehouse and both regional warehouses by the Sprint 4 integrated demo
RO4	Region rules as configuration	A third synthetic region is added by configuration only, with no code change (test)
RO5	Security at release	0 unresolved authorisation defects (cross-region access, role bypass) at the release gate
RO6	Predictable delivery	Each sprint: completion at least 80% of committed points; blocked at most 2 story-days per team; reopened at most 0.25 per completed story
RO7	Onboarding works	Team B completes at least 3 points in Sprint 2 and takes over payments with the idempotency replay test passing
RO8	Achievable scope	Committed points at most the forecast for each sprint; contingency ladder agreed before Sprint 3

6.C Revised strategy (4 marks)

Options considered

Table 38. Redesign options.

Option	Idea	Assessment
1. Team A alone, region rules ad hoc	Keep the original plan with six people	About 33 points in three sprints against 61 needed; payments uncovered. Rejected.
2. Two independent full-stack teams	Each team owns whole slices and its own order code	Duplicate order and payment logic; contract drift; high rework risk. Rejected.
3. Two stream-aligned teams with shared contracts, integration lane and config-based region rules (selected)	Team A: order, stock, pick, dispatch, tracking, support. Team B: payments, returns, refunds, region rules, cart UI, CI	Clear ownership, one integration owner, controlled coupling. Costs onboarding and coordination.

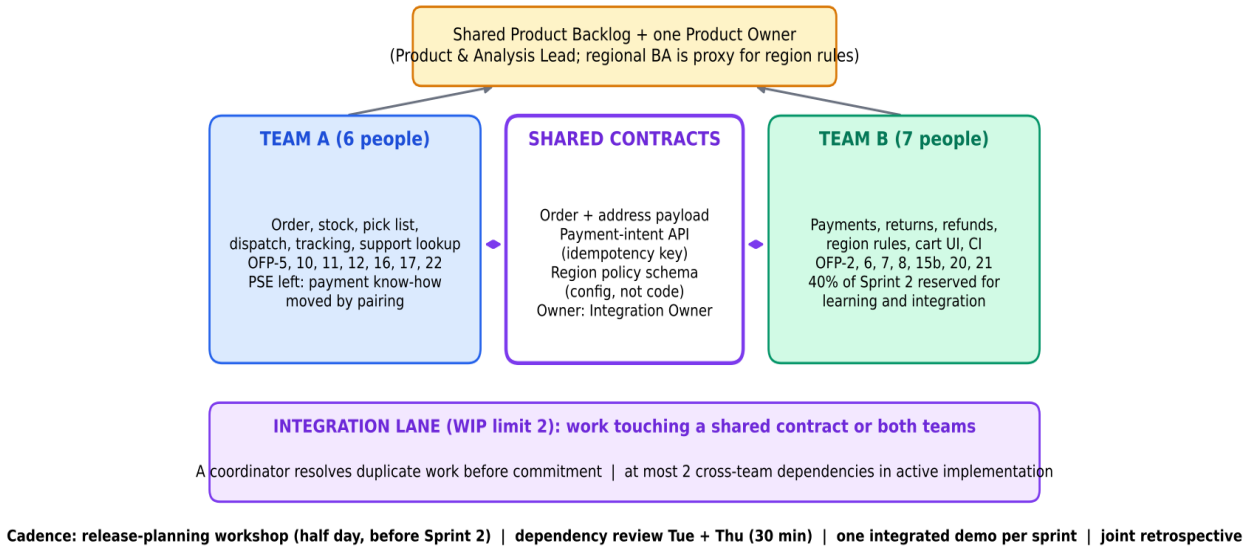


Figure 13. Cross-team coordination design: two teams, shared contracts and an integration lane.

Scaled Agile practices adopted

Table 39. What I adopt and why.

Practice (SAFe-inspired or Lean)	Adopted?	Why it fits this project
Shared product backlog and single Product Owner	Yes	One order of priority for 13 people; a regional BA acts as proxy for region rules
Release planning workshop (half day) instead of a two-day PI planning event	Yes, lightweight	Both teams agree contracts, dependencies and deferrals before Sprint 2
Scrum-of-Scrums style dependency review, Tuesday and Thursday for 30 minutes	Yes	Limits cross-team dependencies (RO2) without daily cross-team meetings
One integrated demo per sprint	Yes	Finds integration and authorisation defects early (see R4)
Integration lane with WIP 2 and a named integration owner	Yes	Shared contracts are the most likely bottleneck; WIP caps coupled work
Region policy as configuration with a schema contract	Yes (XP and architecture practice)	Avoids code branches per region; supports RO4
Full SAFe Agile Release Train, Release Train Engineer, portfolio level	No	Designed for 50 to 125 people; ceremony overhead would exceed benefit for 13
Component teams	No	They create more hand-offs and waiting

Revised backlog and ownership

Table 40. New and re-sequenced stories after the Day 4 change.

ID	New story	Epic	Priority	Pts	Depends on	Team	Sprint
OFP-20	Region tax and shipping rules as configuration. AC: add a region by config only; unknown region rejected	E6	Must	3	OFP-3	B	3
OFP-21	Region-specific refund approval rules. AC: limits and approvers vary by region; cross-region approver rejected	E6	Must	5	OFP-8, 20	B	4
OFP-22	Warehouse-scoped stock, pick lists and access. AC: picker of warehouse X gets 403 on warehouse Y	E6	Must	5	OFP-4, 10	A	3
OFP-23	Regional warehouse onboarding runbook and data load	E6	Could	2	OFP-22	B	Deferred
OFP-11a / 11b	OFP-11 split: status transitions (3) and dispatch screen (2)	E3	Must	3 / 2	OFP-10	A	2
OFP-13, 14	Barcode scan confirmation and low-stock alert	E3, E1	Should	3, 2	OFP-10, 4	-	Deferred
OFP-9	Payment reconciliation export	E2	Should	3	OFP-6	B	Stretch (S4)

Table 41. Revised release plan (committed work). Sprint 4 also holds stretch OFP-9 (3 points) only if the test queue is clear. Team B carries 11 points in Sprint 3, slightly above its proportional share; OFP-7 is the item Team A can absorb.

Sprint	Goal (revised)	Team A	Team B	Pts (A+B)	Forecast
2	Paid order can be picked and dispatched with audited status; payments handed over; Team B onboarded.	OFP-5, OFP-10, OFP-11a, OFP-11b	OFP-2, OFP-15b	11+5=16	16
3	Gateway confirmation, region rules, customer tracking, returns and warehouse-scoped access.	OFP-12, OFP-22	OFP-6, OFP-20, OFP-7	8+11=19	19
4	Controlled region-aware refund, support visibility and a journey demo for all three warehouses; release rehearsal.	OFP-16, OFP-17	OFP-8, OFP-21	6+10=16	20

Deferred after the change: OFP-13 barcode scan confirmation (3), OFP-14 low-stock alert (2) and OFP-23 regional onboarding runbook (2) are removed from the first release, with OFP-9 reconciliation as stretch. Committed total is 51 points against a 55-point forecast, leaving about 7% buffer plus the Sprint 4 slack. Manual pick confirmation with a reason replaces scanning; finance uses the audit-log export for interim reconciliation. Analytics and forecasting stay deferred.

Responsibilities, Lean practices and Jira artefacts

- **Roles.** PL is the Product Owner for both teams; a Team B regional BA owns region rules; Team A's Order API engineer is the Integration Owner with a Team B deputy; one engineer from Team B pairs on payments with Team A before taking over the former PSE responsibilities.
- **Lean controls.** WIP stays 4 / 2 / 2 per team plus integration lane WIP 2; stories are capped at 3 points where they cross a contract; one coordinator removes duplicate work before sprint commitment; carryover stays visible.
- **Jira artefacts.** Epic E6 Regional Operations; components Team-A and Team-B; swimlanes by team plus an integration lane; quick filters "Cross-team" and "Blocked"; a dependency report built from "is blocked by" links; release gate field for authorisation defects; a shared Release 1.0 version. Because the project now has 14 or more users, the free-tier cap (verify) is exceeded; a paid plan or view-only stakeholder access is part of the cost estimate.

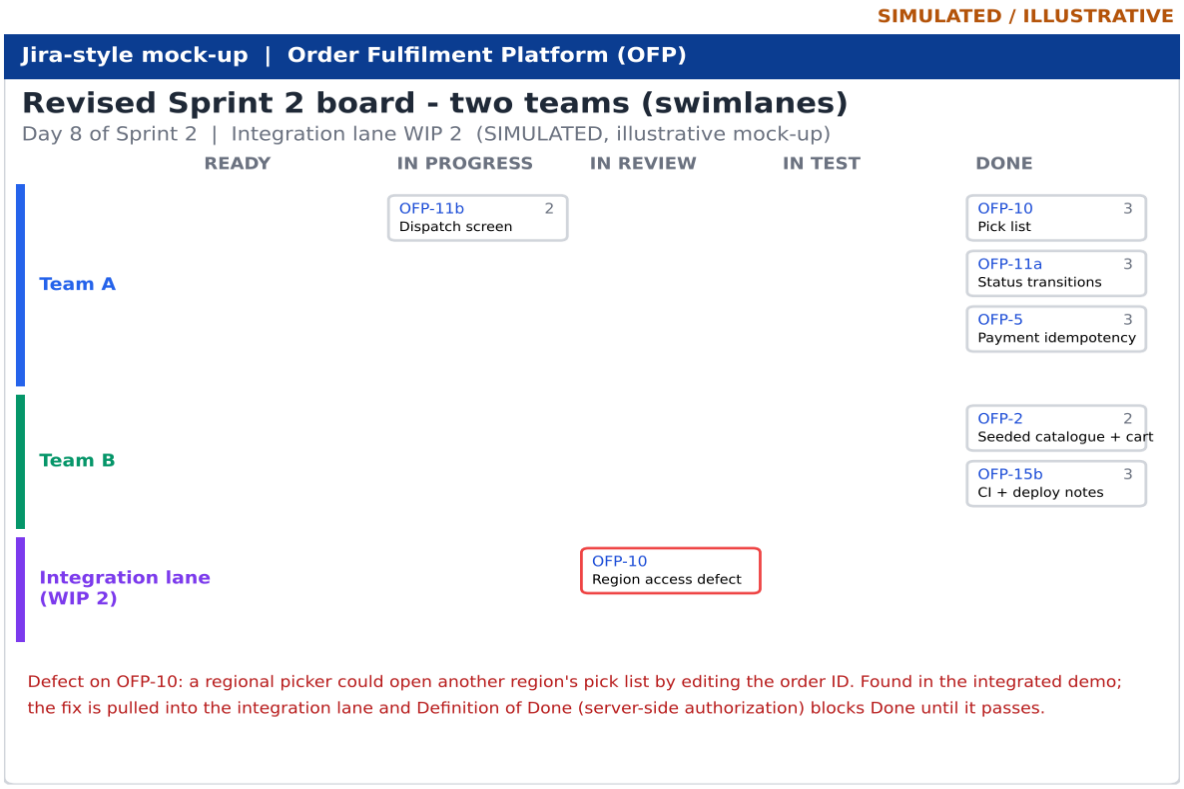


Figure 14. Revised Sprint 2 board with team swimlanes and an integration lane (SIMULATED, illustrative mock-up).

6.D Validate the redesign (2 marks)

Table 42. Seven additional simulated events (at least five required).

#	Day	Simulated event	Revised behaviour
R1	S2-D1	A region requests a special tax and shipping rule.	Policy schema extended; OFP-20 created as configuration; no duplicated order code.
R2	S2-D2	Team B proposes an address payload that conflicts with Team A (field names differ).	Dependency review resolves naming before coding; OFP-11a waits one story-day (blocked 1).
R3	S2-D3	The original payments engineer is unavailable.	OFP-5 handed to Team B's payments engineer, paired with the Order API engineer; the 10-retry idempotency replay test is the executable handover.
R4	S2-D4	Thursday dependency review shows a regional picker can open another region's pick list by changing the order id.	OFP-10 returned from Review (reopened 1); fix pulled into the integration lane; Definition of Done blocks Done until authorisation tests pass.
R5	S2-D5	Gateway sandbox access is delayed again.	Local payment adapter and replay tests keep progress; OFP-6 was already planned for Sprint 3.
R6	S2-D8	Midpoint capacity review: Team B is at its planned 60% availability; 11 of 16 points done.	On track; OFP-13, 14 and 23 stay deferred; stretch OFP-9 is not pulled.
R7	S2-D10	Integrated sprint review with warehouse and finance.	14 of 16 points accepted; OFP-11b carries over (2); pick-to-dispatch shown for the central warehouse.

Table 43. Comparative indicators (all SIMULATED).

Indicator	Original Sprint 1 (7 people, one team)	What-if: original strategy under change (Sprint 2)	Revised strategy (Sprint 2)
Planned / completed points	16 / 13	16 / 8	16 / 14
Completion ratio	81.3%	50.0%	87.5%
Blocked story-days	3	3	1
Reopened stories per completed story	2 / 4 = 0.50	2 / 3 = 0.67	1 / 5 = 0.20
Mean cycle time (days)	7.75	8.33	6.2
Test-queue wait (story-days)	1	2	1
Cross-team dependencies active at once	not applicable	4 (uncontrolled)	at most 2

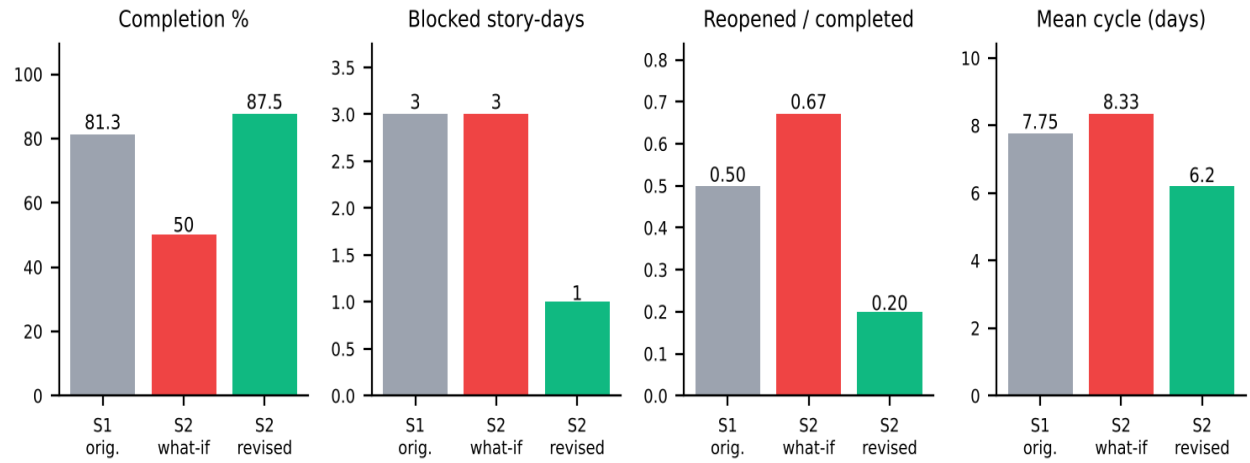


Figure 15. Equivalent indicators for the original sprint, the what-if and the revised strategy (SIMULATED).

The what-if column applies the same event process to a six-person Team A that keeps the original strategy: no shared contract, no integration lane, OFP-11 kept at 5 points and the payments knowledge lost. The three columns involve different teams and scope, so the comparison shows direction and mechanism rather than causal proof. The measured benefit comes from contracts and the integration lane (R2, R4), pairing (R3) and the split of OFP-11 (completion and cycle time).

6.E Defend the decision (2 marks)

Table 44. Benefits and trade-offs.

Benefits	Limitations and trade-offs
Ownership is clear (Team A order and stock, Team B payments, refunds and region rules); the integration owner removes ambiguity. A complete journey is protected: every Must step remains. Authorisation and contract defects were found early by the integrated demo and dependency reviews. Capacity is planned at 1.44x, not 2x, so the forecast is credible.	Onboarding and coordination cost about 15% of focused hours and 40% of Team B's first sprint. Four features (scan confirmation, low-stock alert, onboarding runbook, reconciliation as stretch) leave the first release; pickers use manual confirmation. One integration owner is a single point of failure. Cost rises from about INR 12.6 lakh to INR 20.7 lakh (+64%): Team A 250 person-days plus Team B 210 = 460 x INR 4,500 (assumed rate), plus any paid Jira plan.

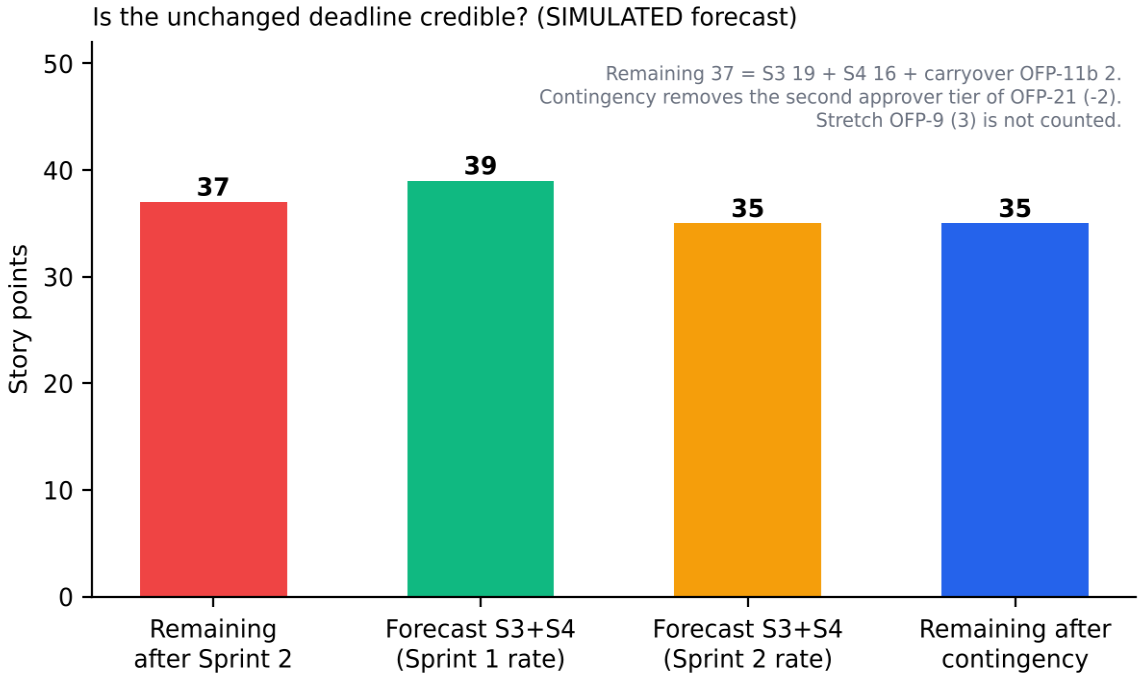


Figure 16. Is the unchanged deadline credible? Remaining committed points against forecast ranges (SIMULATED).

After the simulated Sprint 2, 37 points remain (Sprint 3: 19, Sprint 4: 16, carryover OFP-11b: 2). Using the Sprint 1 rate the forecast for Sprints 3 and 4 is 39 points; using the lower Sprint 2 rate (14 points per 416 effective hours) it is 35. The plan is therefore **feasible but not safe**, which is why a contingency ladder is agreed now:

Table 45. Contingency ladder, agreed before Sprint 3.

Trigger (Sprint 3 review)	Action	Points released
Completion below 80% or forecast gap above 2 points	1. Do not pull stretch OFP-9	0 (already not counted)
Gap persists	2. Remove the second-approver tier from OFP-21 (single approval, still audited)	2
Gap persists	3. Reduce OFP-17 to a smoke-level journey script	1
Authorisation defect unresolved	4. Release gate: do not enable region refunds until fixed; launch with central warehouse plus one region (sponsor decision)	Scope decision

Added people alone would not have rescued the plan, because the redesign relies on explicit contracts, deferral and a lower forecast than the nominal doubling. The deadline is credible only if analytics, forecasting, scan confirmation, the low-stock alert and the onboarding runbook stay deferred.

Task 7 – Individual Technical Reflection (5 marks)

The most significant decision was to protect one complete order-to-refund journey instead of delivering all nine modules. Figure 1 showed that analytics mattered to nobody on day one, while a broken link anywhere between reservation (OFP-4) and refund approval (OFP-8) would have left every user group worse served. Every later decision, from deferring OFP-18 and 19 to deferring OFP-13 and 14 after the change, applied the same test: does the journey still work?

Evidence changed my strategy. I predicted that the QA test queue would be the bottleneck. The simulated Sprint 1 log showed it cost one story-day, whereas three blocked story-days came from a dependency I had missed (OFP-5 needs OFP-4) and two reopened stories came from stories that were not truly Ready (OFP-4, OFP-1). I therefore added the dependency, split 5-point stories (OFP-11 into 11a and 11b), made a negative case part of the Definition of Ready, and added a swarm rule. The staffing change in Task 6 reinforced the lesson that one person owning payments is a delivery risk, so payments work is now paired.

Where the final design remains vulnerable. First, the capacity forecasts rest on one simulated sprint of 13 points, so the 35 to 39 point range could be wrong; the contingency ladder is a response, not a guarantee. Second, a payment sandbox may behave differently from a live gateway, so the idempotency tests prove my contract, not production safety. Third, the integration owner is a single point of failure, and fourth, the 15% coordination overhead and the 60% onboarding availability are assumptions I did not measure.

Ethical concerns. Privacy: all test data are synthetic, Jira tickets carry references rather than customer details, support screens mask personal data, and the Definition of Done forbids real card or customer data in code, logs or tests. Workload: stories are sized relatively and flow indicators describe the team, not individuals; I would not use pick or scan timestamps to rank pickers, and carryover is shown openly instead of being hidden as "partly done". Accessibility: the tracking and support pages must work by keyboard and screen reader and must not rely on colour alone (OFP-12 acceptance criteria and Definition of Done). Transparency: every simulated number is labelled, and the Jira figures are mock-ups, not claims about a real team.

What I would investigate next before using this strategy with a real team: actual order volume and festival peaks (my 1,200 orders per hour is an assumption), how quickly warehouse and finance stakeholders really respond within their 90-minute window, measured gateway latency and failure rates, and the real velocity of the team after two sprints. I would also test whether the WIP limits of 4 / 2 / 2 suit the actual QA capacity and check the accessibility of Jira itself for every team member.

Appendix A – Mapping to the assignment requirements

Table 46. Traceability from the assignment brief to this report.

Requirement	Where addressed
Task 1: questions, evidence, goal, scope, criteria, feasibility	Sections 1.1 to 1.7; Figure 1
Task 2: two approaches, selection, operating model, Lean, tool investigation	Sections 2.1 to 2.5; Figure 2
Task 3: epics, stories, acceptance criteria, priority, estimates, dependencies, DoR/DoD, decisions	Sections 3.1 to 3.7; Figures 3 and 4; OFP_jira_import.csv
Task 4: capacity, sprint goals, dependencies, workflow, WIP, blocked/urgent/unfinished, product goal	Sections 4.1 to 4.6; Figures 5 to 9
Task 5: prediction, event log (12 events), tool updates, indicators, improvement	Sections 5.1 to 5.5; Figures 10 and 11
Task 6 A to E: impact, objectives, revised strategy, validation (7 events), defence	Sections 6.A to 6.E; Figures 12 to 16
Task 7: reflection	Task 7
Sources and tools acknowledgement	Appendix C

Appendix B – Creating the real Jira project and screenshots

1. Create a free Jira Software account and a Scrum project named Order Fulfilment Platform with key OFP.
2. Open Settings, System, External System Import, CSV. Upload OFP_jira_import.csv and map: Summary, Issue Type, Priority label, Story point estimate, Sprint, Description (contains acceptance criteria), Labels, Owner role and the dependency column.
3. Create the four sprints, assign issues as in the Sprint column, and set board column constraints to 4, 2 and 2.
4. Add the "is blocked by" links listed in the Depends on column (the import file lists them as text).
5. Replay the Sprint 1 event log (Table of events 1 to 12) by transitioning issues and flagging OFP-5; export the sprint report and board.
6. Take screenshots of the backlog, sprint board and reports and replace Figures 3, 4, 8 and 14. Keep the label "SIMULATED" in captions.

Appendix C – Acknowledgement of sources and tools

- Schwaber, K. and Sutherland, J. (2020). The Scrum Guide. scrumguides.org.
- Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press; Kanban Guide for Scrum Teams (2021), Scrum.org.
- Poppendieck, M. and T. (2003). Lean Software Development: An Agile Toolkit. Addison-Wesley.
- Beck, K. (2004). Extreme Programming Explained, 2nd ed. Addison-Wesley.
- Scaled Agile, Inc. SAFe framework documentation (scaledagileframework.com), used for the SAFe-inspired practices.
- W3C (2018). Web Content Accessibility Guidelines (WCAG) 2.1, used for the accessibility checks.
- Atlassian, Microsoft (Azure DevOps), GitHub and Trello product documentation and pricing pages, to be checked at the time of use because plans change.
- Tools: Jira Software (target tool), Python with matplotlib and python-docx (figures and document), LibreOffice (layout check).

- Generative AI assistance (Claude, Anthropic) was used to help draft, structure and check this report. All simulated data are constructed and labelled; the student must review and adapt the content to their own project and follow their institution's policy on AI assistance.

Appendix D – Preparation for the verification discussion

Table 47. Likely verification questions.

Possible question	Short answer and where to look
Defend a backlog priority.	OFP-4 before analytics: overselling creates failed orders and refunds. Score is an input; dependencies override it (Section 3.3).
Explain capacity without converting points to hours.	420 nominal and 336 focused hours bound the work; points only rank relative size; 16 is a provisional ceiling revised using the observed 13 points (Section 4.1).
Trace a story through the workflow.	OFP-4: Ready, In Progress (D2), In Review (D6), waited (D7), In Test (D8), defect, In Progress, In Review (D9), Done (D10) (Task 5).
Justify a WIP limit.	In Test 2 because one QA engineer; In Progress 4 for seven people with shared roles (Section 4.5).
Interpret an indicator.	3 blocked story-days: missed target of 2; cause was an unrecorded dependency, not the QA queue.
Which feature was deferred after the change, and why?	OFP-13 scan confirmation, OFP-14 low-stock alert and OFP-23 onboarding runbook: lowest effect on the order-to-refund journey (Section 6.C).
What if a constraint changes again?	Use the contingency ladder (Section 6.E) and re-run the capacity table with new availability.